Machine Learning for Computational Science and Engineering (2026) 2:38
https://doi.org/10.1007/s44379-026-00085-9

RESEARCH

Continual learning for autoregressive PDE surrogates under evolving physical regimes

Hamed Hemati[1] · Binh Duong Nguyen[1] · Stefan Sandfeld[1,2]

Received: 20 May 2026 / Accepted: 14 July 2026
© The Author(s) 2026

Abstract

Deploying machine learning surrogates in scientific simulations faces multifaceted challenges, primary among which is the lack of Continual Learning (CL) capabilities—specifically, the inability to adapt to new physical regimes without significantly degrading performance on prior ones. This is particularly problematic for autoregressive surrogates of time-dependent Partial Differential Equations (PDEs), where small prediction errors can accumulate over long rollouts and new physical regimes overwrite previously learned dynamics. We formulate this adaptation as a CL problem, demonstrating that while standard Experience Replay (ER) is a robust baseline across Advection-Diffusion, Burgers', and Navier-Stokes equations, storing full high-resolution rollouts can be memory-inefficient. To address this, we introduce *Replay-TS*, a temporal-slicing replay strategy that stores compact autoregressive windows sampled across past simulations. Through empirical analysis, we show that Replay-TS exploits the low-frequency spectral redundancy of physical systems to enable *sparse supervision* for rollout steps. By preserving the contiguous historical context and sparsely penalizing the autoregressive target steps, Replay-TS improves retention performance under a fixed memory budget by leveraging higher sample diversity. Replay-TS consistently outperforms standard ER methods across standard 1D and 2D streams, achieving over a 30% MSE reduction in a mixed-physics stream, while remaining architecture-agnostic.

Keywords Scientific machine learning · PDE surrogates · Continual learning

1 Introduction

The success of neural networks as simulation surrogates has been substantial in applications ranging from weather forecasting to material science [1–9]. However, such data-driven models trained on static datasets are limited by their generalization capabilities [10, 11]. Since fixed datasets capture only a fraction of the underlying system's physical possibilities, standard autoregressive models used for time-dependent PDEs are often specialists: they perform well on a specific physical regime — for example, a fixed diffusivity coefficient in the heat equation or a fixed viscosity in the Navier-Stokes equations [12–15] — often fail to predict when governing parameters or equations shift. In other words, when the physical setting evolves, the model effectively becomes "obsolete," and necessitates retraining from scratch. To overcome this limitation via continuous adaptation, they must be able to learn from a simulation stream with temporally varying physical regimes, rather than relying solely on static datasets.

Despite considerable efforts in the *adaptive* and *online* learning literature to mitigate the static nature of offline surrogates (surrogates that are trained once and deployed for prediction), these paradigms serve fundamentally distinct objectives. Adaptive learning frameworks typically focus on a form of fine-tuning or transfer learning to adapt the surrogate to a specific target physical regime or minimize local approximation errors [16, 17]. This means that these methods are unidirectional and prioritize optimization solely for the current physical setting, and do not necessarily preserve

✉ Hamed Hemati
h.hemati@fz-juelich.de

Binh Duong Nguyen
bi.nguyen@fz-juelich.de

Stefan Sandfeld
s.sandfeld@fz-juelich.de

[1] Institute for Advanced Simulations – Materials Data Science and Informatics (IAS-9), Forschungszentrum Jülich GmbH, 52425 Jülich, Germany

[2] Chair of Materials Data Science and Materials Informatics, Faculty 5 – Georesources and Materials Engineering, RWTH Aachen University, 52056 Aachen, Germany

Published online: 08 August 2026

previously learned dynamics. Conversely, *online* or *in-situ* training methods focus on the computational and memory efficiency of the training process itself [18, 19]. For example, instead of storing high-resolution trajectories to disk and generating a large scale dataset, they attempt to train the model with only one simulation at a time without generating a "conventional" dataset. Neither paradigm is explicitly designed to operate over sequentially changing, mutually exclusive physical regimes.

In contrast, the goal of *Continual Learning* (CL) [20, 21] is to *sequentially* learn from a stream of physical experiences—that is, a continuous sequence of datasets arriving over time, where each dataset represents a distinct physical regime. Broadly defined, CL enables sequential adaptation to new physical regimes without suffering from *catastrophic forgetting* [22, 23], which in the context of scientific machine learning can be interpreted as the rapid degradation of performance on previously mastered dynamics. For example, if a surrogate model is initially trained on low-velocity advection processes (the first experience) and then adapted to high-velocity processes (the second experience), it should not lose the ability to simulate the earlier low-velocity regime after adaptation. While this is a well-known paradigm in general machine learning, the physical sciences present a unique constraint: the data is often temporal with a high resolution. Unlike image classification, where samples are individual frames (single snapshots) [24–26], physical simulations are often continuous sequences of evolving states, making the preservation of model performance significantly more challenging.

A standard and popular approach in CL is Experience Replay (ER) [27], where a memory buffer stores past data to rehearse alongside new experiences. However, in autoregressive surrogates for time-dependent PDEs, storing full simulation trajectories can create a memory bottleneck. A single solution trajectory in a simulation can consist of hundreds to thousands of high-resolution time steps, occupying a massive memory footprint compared to standard static datasets. As a result, a fixed buffer filled with full-length trajectories quickly saturates, forcing a difficult trade-off: the learning system can either retain the few full temporal trajectories for all encountered physical regimes, or completely remove data from the older physical regimes to enable storage of more samples with higher diversity for the newer ones. This results in a loss of stored physical regime, which will potentially leave the model more vulnerable to forgetting earlier physical dynamics.

In this work, we address this memory bottleneck by challenging the assumption that preserving performance in learned physical operators via replaying past trajectories requires full-trajectory storage. We argue that a broad class of physical systems— specifically those governed by dissipative or low-frequency dynamics—exhibits smooth spectral properties. In these systems states evolve gradually without abrupt, high-frequency oscillations, and therefore full trajectories contain temporal redundancy. Consequently, the essential dynamics required to learn the evolution operator can be captured through sub-sequences of the simulation's trajectory than the entire history. Based on this insight, our specific contributions are as follows:

- We define the adaptation of autoregressive surrogates to changing physical parameters as a CL problem, and provide empirical evidence of generalization failure in physical regime-shifts.
- We provide a comprehensive benchmark of standard CL methods across 1D and 2D PDE families, including Advection-Diffusion, Burgers', and Navier-Stokes equations, and highlight the failure modes of the static learning approach.
- We introduce *Temporal Slicing*, a memory-efficient replay strategy that exploits spectral redundancy to decompose trajectories into information-dense sub-sequences. We demonstrate that this approach improves the diversity of retained physical regimes by enabling higher sample storage capacity, and outperforms standard baselines under fixed memory budgets.

2 Related work

The transition from static physical simulators to continually adapting surrogates requires models that can not only approximate complex dynamics but also evolve over time to enable predictions on unseen physical conditions. Our work addresses the intersection of two topics: data-driven surrogates for scientific simulation and continual learning under distribution shifts. We review the related work for each topic in the following sub-sections.

2.1 Data-driven surrogates

One way to categorize machine learning for scientific problems is by the direction of inference: forward modeling and inverse problems [28].

Forward modeling Forward modeling addresses the causal mapping from defined parameters to physical behavior. While traditional approaches rely on computationally expensive numerical integration (e.g., Finite Element or Finite Volume methods) [29, 30], data-driven forward modeling for time-dependent PDE aims to approximate the solution operator $\mathcal{G} : (\mathbf{u}_0, \boldsymbol{\mu}) \mapsto \mathbf{u}(t)$. By learning this mapping, neural networks act as efficient surrogates for classical solvers, significantly accelerating the evaluation of physical states.

Inverse problems Inverse problems attempt to reverse the causal forward map $\mathcal{G}$ defined above, and generally fall into

two distinct classes based on their objective and dimensionality. The first is the classical mathematical inverse problem, often termed *parameter identification*. Here, given a set of physical observations $\mathbf{u}(t)$, the goal is to infer the underlying low-dimensional physical coefficients $\boldsymbol{\mu}$ (such as viscosity or diffusivity) that govern the PDE. The second class, frequently encountered in the natural sciences and materials engineering, involves learning the high-dimensional *inverse map* itself. This requires synthesizing complex spatial topologies, geometries, or full initial conditions $\mathbf{u}_0$ that satisfy specific target properties [31, 32]. Because this mapping is highly non-linear, high-dimensional, and often ill-posed (as multiple structures can yield the same outcome), recent approaches frame it as a generative task. Architectures such as Diffusion Models [33], Generative Adversarial Networks (GANs) [34], or Reinforcement Learning [35] are utilized to sample from the distribution of valid physical structures [36–38].

While inverse problems focus on parameter estimation or structure generation, our work is fundamentally concerned with the opposite direction: enabling continual learning for forward modelling in environments where the physical parameters $\boldsymbol{\mu}$ themselves evolve sequentially over time. One way to categorize forward modeling for time-dependent PDE is as below:

One way to categorize forward modeling for time-dependent PDE simulations is as below:

1. **Physics-Informed Solvers:** Physics-Informed Neural Networks (PINNs) [39–41] solve the forward problem by embedding the PDE residuals directly into the loss function. This allows them to approximate the solution without labeled simulation data. However, standard PINNs are typically transductive, which means they require re-optimization for every new instance of boundary conditions, which limits their utility for real-time applications.
2. **One-Shot Operators:** To enable rapid inference, "One-Shot" architectures like DeepONets [42–44], and Fourier Neural Operators (FNO) [13] learn a mapping from the initial condition directly to the solution at any time step t. While powerful, these approaches can struggle with long-time horizons where temporal dependencies become highly complex.
3. **Autoregressive (AR) Modeling:** The dominant paradigm for long-horizon evolution is iterative time-stepping [45–48]. Here, a neural network $\mathcal{N}_\theta$ learns the discrete transition operator $\mathbf{u}_{t+1} = \mathcal{N}_\theta(\mathbf{u}_t; \boldsymbol{\mu})$. Our work focuses on this approach due to its temporal generalization capability.

2.2 Continual learning

Continual Learning (CL) addresses the challenge of learning from non-stationary data streams of learning experiences, where the underlying data distribution shifts over time, without *catastrophic forgetting* [20]. Unlike static datasets, data streams consist of a sequence of experiences arriving over time. The goal is to accumulate learning experience as the learner is provided with a new experience rather than overwrite it. While CL is becoming a well-established paradigm in domains like computer vision [49, 50], its application to physics-based surrogates remains a nascent and largely under-explored frontier.

Standard CL methods are typically categorized into three main families, each tackling the stability-plasticity dilemma [51] differently:

- **Regularization Methods:** Methods such as *Elastic Weight Consolidation (EWC)* [52] identify parameters crucial for previous experiences and penalize significant changes to them. Essentially, regularization-based approaches introduce a mechanism that pulls weights back towards their optimal values for old experiences while allowing less critical weights to adapt to new data [53–55].
- **Parameter Isolation:** Methods like PackNet [56] dedicate specific sub-networks or masks to distinct experiences. By structurally segregating knowledge, these models prevent interference but often require the model size to grow with the number of experiences or leave capacity under-utilized [57, 58].
- **Experience Replay (ER):** ER-based methods are often considered the simplest yet the most effective methods for overcoming data shifts. ER maintains an "episodic memory" of past data [59]. By mixing buffered examples from previous experiences with the current training stream, the model approximates the joint distribution of all experiences up to the current point of training [60, 61]. Advanced variants explicitly constrain optimization to prevent "optimization conflicts": *Gradient Episodic Memory (GEM)* [62] projects the current gradient so that it does not increase the loss on stored samples, mathematically ensuring stability. Its efficient successor called *Averaged-GEM (A-GEM)* [63], relaxes this by constraining the gradient based on the average of random memory batches which significantly reduces computational overhead while preserving the benefits of replay.

In the domain of scientific machine learning, existing literature has largely prioritized Transfer Learning over Continual

Learning. Studies have repeatedly demonstrated that operator learners, such as DeepONets and FNOs or Transformer-based models, can be effectively fine-tuned for new physical regimes [64–67]. However, these approaches are typically unidirectional: optimizing for a target domain degrades performance on the source domain, which leads to significant forgetting rather than experience accumulation.

Attempts to mitigate this forgetting have often relied on specific architectural modifications. For instance, in the context of PINNs, *Incremental PINNs (iPINNs)* [68] train separate subnetworks for different equations, effectively sidestepping the forgetting problem by continually expanding model capacity. More recently, approaches such as the *Neural Combinatorial Wavelet Neural Operator (NCWNO)* [69] have been proposed as "foundation models". These methods achieve robustness against catastrophic forgetting by utilizing specialized architectures, specifically by using gated structures with local wavelet experts and memory-based ensembling.

While architectural innovations like the *NCWNO* are promising, they do so at the cost of modularity and require researchers to replace the entire surrogate model with specialized gated structures and local wavelet experts. Our work pursues an orthogonal direction by focusing entirely on the *learning strategy*. Because *Temporal Slicing* is an architecture-agnostic memory mechanism, a direct empirical comparison with custom-built CL approaches falls outside our scope. Rather, our objective is to demonstrate that memory-efficient replay can improve model's CL capabilities widely adopted, unmodified backbones (e.g., standard FNOs and U-Nets) through better memory management alone (Fig. 1).

3 Problem formulation and analysis

In this section, we first formalize the learning of time-dependent PDEs as an autoregressive learning problem. Then, we empirically show that a physical parameter shift, leads to generalization failures. Finally, in order to address this issue, we define the Continual Learning (CL) objective to address the failure modes. For a comprehensive summary of the mathematical symbols and variables used throughout this manuscript, please refer to Table 11 in Appendix B.

3.1 Autoregressive modeling of physical dynamics

Let $\Omega \subset \mathbb{R}^d$ be a bounded spatial domain and $\mathcal{T} = [0, T]$ be a temporal interval. We consider a system governed by a time-dependent PDE of the general evolution form:

$$\frac{\partial \mathbf{u}}{\partial t} + \mathcal{N}[\mathbf{u}; \boldsymbol{\mu}] = 0, \quad \mathbf{x} \in \Omega, t \in \mathcal{T}, \tag{1}$$

where $\mathbf{u}(\mathbf{x}, t) \in \mathbb{R}^{d_u}$ represents the physical state field, and $\mathcal{N}[\cdot; \boldsymbol{\mu}]$ is a possibly non-linear spatial differential operator parameterized by a vector of physical coefficients $\boldsymbol{\mu} \in \mathcal{P} \subset \mathbb{R}^p$ (e.g., the advection velocity or diffusivity).

For computational tractability, we assume the spatial domain is discretized into a grid of N_x nodes. Consequently, the continuous state $\mathbf{u}(\cdot, t)$ is mapped to a high-dimensional

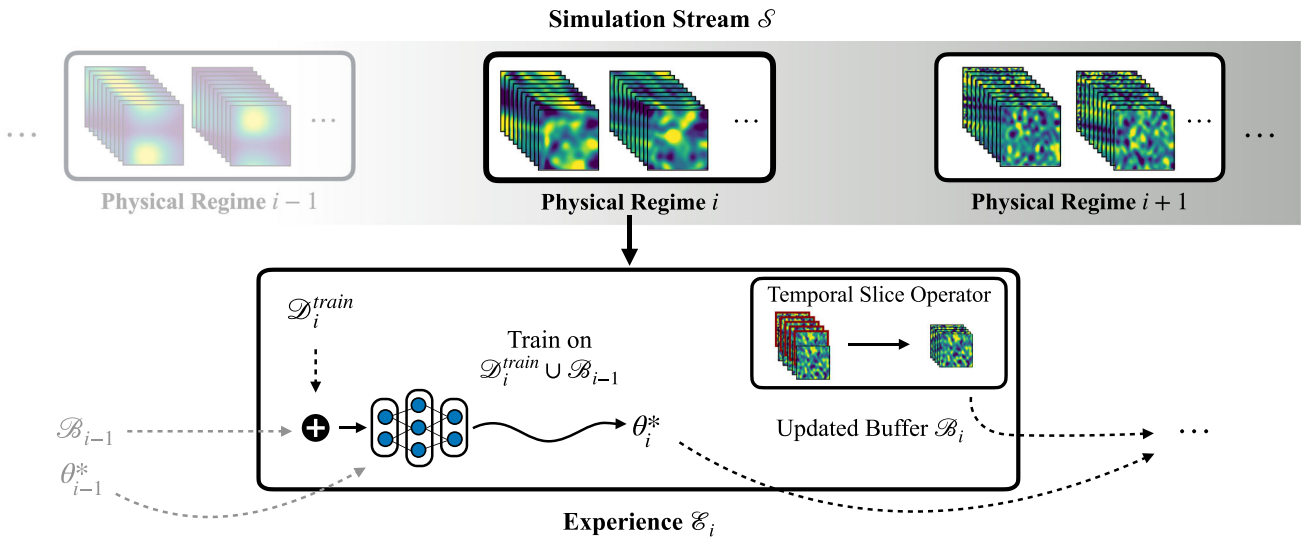


Fig. 1 Schematic overview of the temporal slicing method. The simulation stream $\mathcal{S}$ generates data for a sequence of physical regimes. Each stack represents a full continuous simulation trajectory in the form of a spatiotemporal field, where individual frames correspond to the 2D spatial state at consecutive time steps. In experience $\mathcal{E}_i$, the model is trained on a mixture of samples from the i-th physical regime and samples from the previous state of the replay buffer ($\mathcal{B}_{i-1}$) to prevent forgetting of earlier dynamics. Since the memory budget is fixed, a *Slice Operator* decomposes the simulation trajectories in the current physical regime into compact temporal sub-sequences to maximize the diversity of samples in the updated replay buffer $\mathcal{B}_i$

vector $\mathbf{u}_t \in \mathbb{R}^{N_x \times d_u}$. A discrete simulation trajectory τ is defined as a sequence of K_τ stored states:

$$\tau = \{\mathbf{u}_0, \mathbf{u}_1, \ldots, \mathbf{u}_{K_\tau - 1}\}, \tag{2}$$

where $\mathbf{u}_t \approx \mathbf{u}(\cdot, t_0 + t\Delta t)$.

Our objective is to learn a parametric surrogate model f_θ, parameterized by weights θ, that approximates the discrete evolution operator. Because stable numerical prediction of time-dependent PDEs can benefit from short temporal histories, we use an autoregressive context window of length L_{hist}. The model predicts the next state from the previous L_{hist} snapshots:

$$\hat{\mathbf{u}}_{t+1} = f_\theta(\mathbf{u}_{t-L_{\text{hist}}+1:t}), \tag{3}$$

$$\text{with} \quad \mathbf{u}_{a:b} := (\mathbf{u}_a, \mathbf{u}_{a+1}, \ldots, \mathbf{u}_b). \tag{4}$$

The desired approximation is therefore

$$\left\| \hat{\mathbf{u}}_{t+1} - \Phi_{\Delta t}(\mathbf{u}_{t-L_{\text{hist}}+1:t}; \boldsymbol{\mu}) \right\| \to 0, \tag{5}$$

where $\Phi_{\Delta t}$ denotes the ground-truth numerical time-stepper.

In a standard static or "offline" training setting, we assume a fixed distribution of physical parameters $\boldsymbol{\mu} \sim P(\boldsymbol{\mu})$ and find the optimal parameters θ^* by minimizing the expected cumulative rollout error. To stabilize autoregressive training, it is standard practice to employ the "*pushforward trick*" [46]. In the *pushforward trick*, the model predicts L_{pf} steps ahead during training and adds each prediction step to the context window. Consequently , the optimization accounts for the recursive dependency on previous predictions over this horizon:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \, \mathbb{E}_{\boldsymbol{\mu} \sim P} \mathbb{E}_{\tau \sim \mathcal{D}} \mathbb{E}_t \left[\frac{1}{L_{\text{pf}}} \sum_{k=1}^{L_{\text{pf}}} \ell(\hat{\mathbf{u}}_{t+k}, \mathbf{u}_{t+k}) \right], \tag{6}$$

where the prediction at rollout step $t + k$ is computed recursively as

$$\hat{\mathbf{u}}_{t+k} = f_\theta(\tilde{\mathbf{u}}_{t+k-L_{\text{hist}}:t+k-1}). \tag{7}$$

The initial history is provided by the ground-truth trajectory up to the current anchor step t, i.e., $\tilde{\mathbf{u}}_j = \mathbf{u}_j$ for $0 \le j \le t$, while subsequent inputs are generated autoregressively, i.e., $\tilde{\mathbf{u}}_j = \hat{\mathbf{u}}_j$ for $j > t$.

3.2 Sensitivity of static surrogates to physical shifts

Deploying PDE surrogates remains constrained by the assumption that the governing operator $\mathcal{N}$ remains static. In real-world applications, models may need to generalize across variations in the underlying physics. We distinguish between three distinct classes of physical distribution shifts specific to time-dependent PDEs:

1. **Intra-Regime Shift (Variations in $\mathbf{u}_0$):** The governing physical laws and coefficients remain constant, but the *distributional family* of the initial conditions varies. Generalization here requires the model to correctly predict the system's evolution across diverse, unseen initial state distributions. For example, a model trained on data with sinusoidal initial conditions, is tasked with predicting the same physics starting from Gaussian Random Fields (GRFs), which exhibit different multi-scale characteristics.
2. **Inter-Regime Shift (Parametric Variations in $\boldsymbol{\mu}$):** The structure of the governing equation remains the same, but the physical coefficients change (e.g., variations in advection velocity, diffusivity or viscosity). This shifts the system's underlying dynamics, requiring the model to capture a new set of physical behaviors.
3. **Fundamental Equation Shift (Structural Changes):** The underlying governing laws themselves change fundamentally (e.g., a transition from Linear Advection-Diffusion to the non-linear Burgers' equation). This represents the most severe shift, where the model must adapt to a completely new mathematical operator structure rather than a simple parametric shift.

Example: empirical analysis of generalization failure

To demonstrate the sensitivity of autoregressive models to Inter-Regime shifts as an example, we define a two-dimensional parameter space $\mathcal{P}$ for the 1D Advection-Diffusion equation, spanned by advection velocity $a \in [0.3, 0.8]$ and diffusivity $\nu \in [0.001, 0.01]$.

$$\frac{\partial \mathbf{u}}{\partial t} + a \frac{\partial \mathbf{u}}{\partial x} = \nu \frac{\partial^2 \mathbf{u}}{\partial x^2}. \tag{8}$$

We train a baseline surrogate f_{θ^*} to convergence on two source regimes,

$$\boldsymbol{\mu}_{\text{src}} \in \{(0.3, 0.001), (0.8, 0.001)\}, \tag{9}$$

and evaluate the MSE loss on all points in the discretized parameter space:

$$\mathcal{L}(\theta^*; \mathcal{D}_{\boldsymbol{\mu}}) \quad \forall \boldsymbol{\mu} \in \mathcal{P}. \tag{10}$$

As illustrated in Fig. 2, the model's reliability is highly localized to the points in the parameter space that were used for training the model. We observe that as the evaluation parameters drift away from the source regimes $\boldsymbol{\mu}_{\text{src}}$, the prediction error increases. While the severity of this degradation

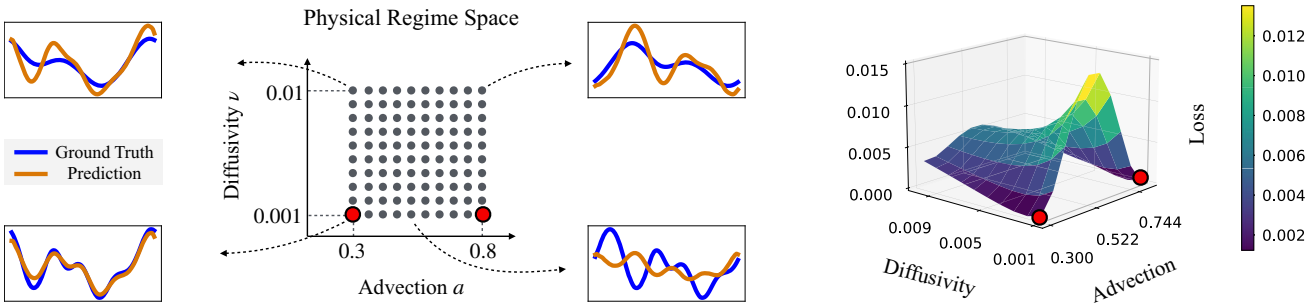


Fig. 2 Impact of inter-regime shifts (parametric variations) on model's prediction. **(Left)** The physical regime space is spanned by the *advection velocity* a and *diffusivity* ν parameters. The red dots in the grid ($\boldsymbol{\mu}_{\text{src}}$) represent the physical regimes used to generate data for pre-training the model, while the gray dots represent unseen physical regimes used for evaluation only. The four panels show the final snapshots of the ground truth and the pre-trained model's predictions in one seen and three unseen physical regimes. The prediction for the test trajectory in $\boldsymbol{\mu}_{\text{src}} = (0.3, 0.001)$ aligns well with the ground truth, however, the predictions in the unseen physical regimes exhibit significant phase and amplitude errors. **(Right)** The loss landscape of the pre-trained model over the entire physical regime space. The sharp rise in error away from $\boldsymbol{\mu}_{\text{src}}$ demonstrates the model's inability to generalize to new physics without adaptation

varies, depending on whether the shift is in advection velocity a or diffusivity ν, the pattern is consistent: the static model fails to generalize. This happens because the network specializes in the specific dynamics it was trained on. For example, it learns to propagate waves at a specific speed a. When tested on a regime with velocity $a' \neq a$, these learned features effectively become invalid, resulting in significant phase misalignment and amplitude errors. This confirms that static models are insufficient for dynamic environments, therefore the surrogate needs to be able to adapt to the evolving physical parameters in such settings.

3.3 Continual learning formulation

We frame the adaptation of autoregressive PDE surrogates as a CL problem. Instead of a static dataset, the learner is exposed to a sequential stream of M experiences (also referred to as tasks), denoted as $\mathcal{S} = \{\mathcal{E}_1, \ldots, \mathcal{E}_M\}$.

Each experience $\mathcal{E}_i$ corresponds to a distinct physical regime governed by an evolution operator Φ_i and characterized by a parameter set $\boldsymbol{\mu}_i \in \mathcal{P}_i$. This formulation accommodates both parametric shifts within a single equation family (where the structural form of Φ_i is constant but $\boldsymbol{\mu}_i$ varies) and fundamental structural shifts (where the underlying PDE Φ_i itself changes). The train and test sets for experience i are given by $\mathcal{D}_i^{\text{train}}$ and $\mathcal{D}_i^{\text{test}}$ accordingly, where $\mathcal{D}_i^* = \{\tau^{(n)}\}_{n=1}^{N_{\text{traj}}}$. The trajectories in the dataset are generated via the corresponding forward operator $\Phi_i(\cdot; \boldsymbol{\mu}_i)$.

During training on experience $\mathcal{E}_i$, the model has access only to the current training dataset $\mathcal{D}_i^{\text{train}}$. The optimization objective is to minimize the loss on the current physical regime while adhering to a stability constraint that prevents performance degradation on all previous physical regimes $\mathcal{E}_{j<i}$ (avoiding Catastrophic Forgetting). Formally, for the i-th experience, we try to optimize:

$$\begin{aligned} \theta_i^* &= \underset{\theta}{\operatorname{argmin}}\, \mathcal{L}(\theta; \mathcal{D}_i^{\text{train}}) \\ \text{s.t.} \quad \mathcal{L}(\theta; \mathcal{D}_j^{\text{train}}) &\leq \mathcal{L}(\theta_j^*; \mathcal{D}_j^{\text{train}}) + \epsilon, \qquad \forall j < i, \end{aligned} \tag{11}$$

where θ_j^* denotes the optimal parameters previously found when training on experience j, ϵ is a tolerance margin representing acceptable forgetting, and $\mathcal{L}(\theta; \mathcal{D})$ denotes the expected cumulative rollout error over the trajectories in dataset $\mathcal{D}$. Simply put, if this inequality holds, the model updates its weights θ to learn the new physics i while ensuring its prediction error on any past regime j will not exceed its original optimal error on that regime by more than ϵ.

Evaluating the final model θ_M requires assessing performance across the test sets of all seen regimes. We utilize the Mean Squared Error (MSE_i) for absolute fidelity and the Relative L_2 Error (Rel-$L_{2(i)}$) for scale-invariant loss measurement:

$$\text{MSE}_i = \mathbb{E}_{\tau \sim \mathcal{D}_i^{\text{test}}} \left[\frac{1}{H - L_{\text{hist}} + 1} \sum_{t=L_{\text{hist}}}^{H} \left\| \mathbf{u}_t - \hat{\mathbf{u}}_t \right\|_2^2 \right], \tag{12}$$

$$\text{Rel-}L_{2(i)} = \mathbb{E}_{\tau \sim \mathcal{D}_i^{\text{test}}} \left[\frac{\left(\sum_{t=L_{\text{hist}}}^{H} \left\| \mathbf{u}_t - \hat{\mathbf{u}}_t \right\|_2^2 \right)^{1/2}}{\left(\sum_{t=L_{\text{hist}}}^{H} \left\| \mathbf{u}_t \right\|_2^2 \right)^{1/2}} \right]. \tag{13}$$

Here, $H \leq K_\tau - 1$ denotes the final temporal index of the sequence used for evaluation. We distinguish this full evaluation sequence length from the shorter pushforward horizon L_{pf} used during training.

To satisfy the stability constraint and approximate the joint distribution of all seen experiences in an implicit manner, *Experience Replay* (ER) is commonly employed to store and rehearse historical data. However, for PDE surrogates, this

strategy encounters a memory bottleneck. Storing the necessary high-dimensional solution trajectories from previous regimes ($\mathcal{D}_{j<i}$) to effectively mitigate forgetting is computationally prohibitive given standard hardware constraints. This limitation necessitates the memory-efficient strategy we propose in Section 4.

4 Proposed method

We propose a method to overcome the memory-specific bottleneck that limits the efficiency of standard Experience Replay (ER) for PDE simulation data. Through spectral analysis, we demonstrate that standard simulation trajectories exhibit significant temporal redundancy. Based on this, we introduce *Temporal Slicing*, a sampling strategy that exploits this redundancy to maximize the diversity of samples in different physical regimes stored in a buffer with a fixed memory budget.

4.1 The bottleneck in trajectory replay

Experience Replay mitigates forgetting by interleaving current training data with samples stored in a memory buffer $\mathcal{B}$. In autoregressive PDE surrogates, however, the atomic supervised training object is not an isolated snapshot but an autoregressive transition window. Because we employ the pushforward trick during training, a valid training item anchored at time t for a trajectory τ must contain both the history context window and the complete pushforward target horizon:

$$z_t = \left(\mathbf{u}_{t-L_{\text{hist}}+1:t},\ \mathbf{u}_{t+1:t+L_{\text{pf}}}\right), \quad \text{where } t = L_{\text{hist}}-1, \ldots, K_\tau - L_{\text{pf}} - 1. \tag{14}$$

Thus, replay must preserve enough temporal context to reconstruct the initial model input as well as the target states for the entire pushforward sequence. A standard implementation stores complete simulation trajectories $\tau \in \mathbb{R}^{K_\tau \times N_x \times d_u}$. When training on a new physical regime $\mathcal{E}_i$, the objective is augmented with a replay term:

$$\mathcal{J}(\theta) = \mathcal{L}(\theta; \mathcal{D}_i^{\text{train}}) + \lambda \mathbb{E}_{\tau\sim\mathcal{B}}\left[\mathcal{L}_{\text{roll}}(\theta;\tau)\right], \tag{15}$$

where λ controls the stability-plasticity trade-off and $\mathcal{L}_{\text{roll}}$ denotes the rollout loss computed from valid autoregressive windows in the stored trajectory.

However, time-dependent PDE data is often temporally dense. A single solution trajectory typically consists of K_τ high-resolution stored snapshots (often $K_\tau \sim 10^3$), where each step represents a complete spatial state vector. To compare replay methods under a fair memory constraint, we distinguish between three quantities:

- **Trajectory length (K_τ):** the number of snapshots stored in one full simulation trajectory.
- **Snapshot capacity (S_{total}):** the maximum number of state snapshots that the replay buffer can store. This is the fixed hardware-level memory budget used for fair comparison.
- **Replay item:** the specific sub-trajectory sampled from the buffer for a training step. It consists of the historical context window plus the required pushforward target window (which is retained in full for standard replay, or sub-sampled as slices for Temporal Slicing).

For standard ER, the maximum number of full trajectories that can be stored in a memory with a snapshot capacity of S_{total} is therefore:

$$|\mathcal{B}|_{\text{traj}} = \left\lfloor \frac{S_{\text{total}}}{K_\tau} \right\rfloor. \tag{16}$$

As the learner encounters a sequence of distinct physical regimes $\{(\Phi_1, \boldsymbol{\mu}_1), (\Phi_2, \boldsymbol{\mu}_2), \ldots\}$, a small fixed-capacity buffer can suffer from *physical regime collapse*. Because each full trajectory consumes K_τ snapshots, only a small number of trajectories can be retained. When new regimes arrive, older trajectories must be discarded to free memory. The resulting buffer may contain too few initial conditions, too few temporal contexts, or too few physical regimes to approximate the joint distribution of previously seen dynamics. This lack of replay diversity leaves the surrogate vulnerable to catastrophic forgetting.

4.2 Spectral analysis of temporal redundancy in subsampled fields

We challenge the assumption that retaining the *entire* sequence of K_τ snapshots is strictly necessary to preserve operator performance. In time-dependent PDEs, the information required to define the local physical evolution often exhibits temporal redundancy. We can formalize this redundancy and its limits through the lens of the Nyquist-Shannon sampling theorem.

To establish the bounds of this redundancy, we analyze an extreme scenario: learning exclusively from temporally *subsampled* fields. If we strictly subsample a trajectory by a factor of $s \geq 1$, the effective time step jumps to $s\Delta t$. This reduces the coarse Nyquist limit to $f_{\text{nyq}}^{(s)} = 1/(2s\Delta t)$. While the Nyquist limit dictates the theoretical boundary for *perfect* signal reconstruction, neural networks can sometimes exceed these classical limits by leveraging learned physical priors. Nevertheless, this spectral boundary provides a measurable framework to diagnose when a subsampled simulation fundamentally aliases, depriving the model of the true underlying dynamics.

To quantify this temporal frequency distribution, we compute the spatially averaged Temporal Energy Spectral Density (ESD). Let $\hat{\mathbf{u}}(\mathbf{x}, f)$ denote the temporal Fourier transform of the state field at spatial location $\mathbf{x}$ for frequency f. The ESD is defined as:

$$E(f) = \frac{1}{|\Omega|} \int_{\Omega} |\hat{\mathbf{u}}(\mathbf{x}, f)|^2 \, d\mathbf{x}. \tag{17}$$

To empirically demonstrate this, we contrast the ESD of two 1D linear PDEs on a periodic domain, both simulated at $\Delta t = 0.02\,\text{s}$ ($f_{\text{nyq}} = 25\,\text{Hz}$):

1. **Low-Frequency Dynamics (Advection-Diffusion):** $\frac{\partial \mathbf{u}}{\partial t} + a\frac{\partial \mathbf{u}}{\partial x} = \nu \frac{\partial^2 \mathbf{u}}{\partial x^2}$ (with $a = 1, \nu = 0.03$). This dissipative system concentrates its energy at low frequencies ($\sim$ 1–3 Hz).
2. **High-Frequency Dynamics (Advection-Dispersion / Linear KdV):** $\frac{\partial \mathbf{u}}{\partial t} + a\frac{\partial \mathbf{u}}{\partial x} = -\beta \frac{\partial^3 \mathbf{u}}{\partial x^3}$ (with $a = 4, \beta = 8 \times 10^{-4}$). This dispersive system is dominated by faster oscillations ($\sim$ 8–11 Hz).

As illustrated in Fig. 3, applying a subsampling factor of $\gamma = 4$ drops the coarse Nyquist limit to $f_{\text{nyq}}^{(4)} = 6.25\,\text{Hz}$. The Advection-Diffusion system's energy remains entirely below this threshold, avoiding aliasing. In contrast, the dominant energy of the Advection-Dispersion system exceeds 6.25 Hz; therefore, the coarse sequence fundamentally aliases the true dynamics.

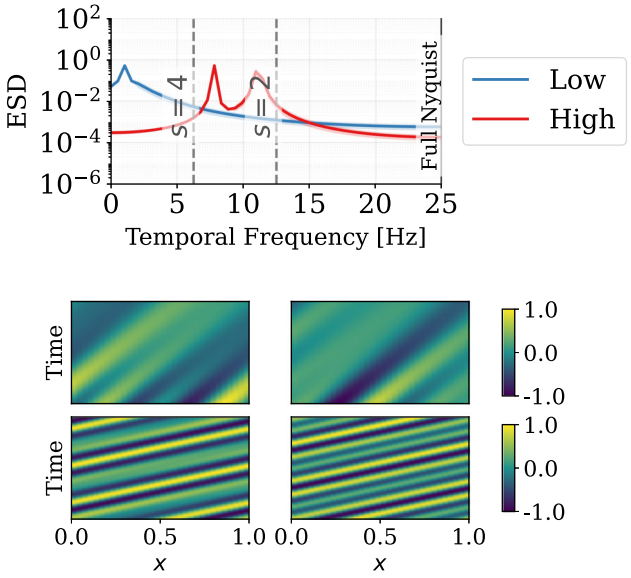


Fig. 3 Temporal spectral density and field dynamics. **Top:** Averaged power spectrum of the dissipative Advection-Diffusion (blue) and dispersive Advection-Dispersion (red) systems. Vertical lines indicate Nyquist limits at native resolution (solid) and under temporal subsampling $s \in \{2, 4\}$ (dashed). At $s = 4$, the high-frequency system aliases, while the low-frequency system remains resolved. **Bottom:** Two random examples of the spatiotemporal evolutions of the state fields (cropped along the time dimension for visualization purposes) in both systems (top: low-freq., bottom: high-freq.)

This spectral distinction directly dictates the surrogate model's predictive limits (Fig. 4). When we train an autoregressive Fourier Neural Operator (FNO) exclusively on these subsampled trajectories to test their learnability, increasing the subsampling factor from $\gamma = 1$ to $\gamma = 4$ barely impacts the full-trajectory reconstruction MSE for the dissipative system. The temporal compression is effectively "free" because the underlying operator remains fully learnable. Conversely, for the high-frequency dispersive system, the aliased coarse data frustrates the learning process. As a result, the model cannot deduce the intermediate dynamics, causing the reconstruction error to exponentially compound.

This analysis establishes the theoretical justification for our memory strategy by probing the inherent "learnability" of temporally subsampled fields. Strict subsampling represents an extreme case: the model is forced to deduce transition dynamics across massive temporal gaps without any intermediate context. The fact that the surrogate can perfectly learn the dissipative operator from this severely coarse data without aliasing proves that the intermediate temporal states contain redundant information. It naturally follows that if pure subsampling is learnable, providing the model with dense intermediate steps but only *sparsely penalizing* the target steps, as we propose in our practical *Temporal Slicing* method, can be sufficient to preserve the physical dynamics.

4.3 Temporal slicing

Building on the insight that sparse temporal signals are sufficient to capture dynamics, we introduce *Temporal Slicing* (*Replay-TS*).

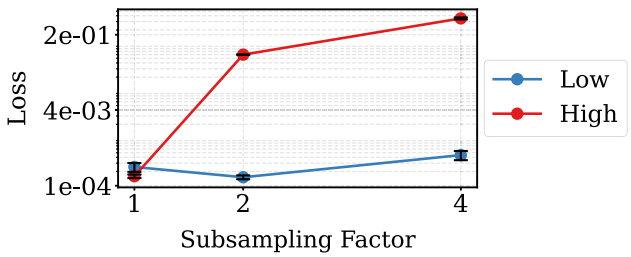


Fig. 4 Impact of temporal slicing on predictability. Full-resolution learning MSE of an FNO trained on subsampled trajectories. The low-frequency Advection-Diffusion system (blue) remains stable and learnable up to a slice factor of $s = 4$. Conversely, the high-frequency Advection-Dispersion system (red) fundamentally aliases at $s \geq 2$, frustrating the learning process and causing an significant growth in the loss (averaged over 5 seeds)

Differentiation from standard sequence subsampling It is critical to distinguish Temporal Slicing from generic trajectory subsampling or frame-skipping commonly utilized in reinforcement learning or video processing [70, 71]. In standard sequence learning, frames are dropped uniformly from the input, effectively increasing the prediction time-step to a macro-step (e.g., $k\Delta t$). For autoregressive PDE surrogates, however, artificially increasing the time-step often violates numerical stability limits (such as the Courant–Friedrichs–Lewy (CFL) condition) and degrades the model's ability to capture local temporal derivatives.

Replay-TS circumvents this limitation through *dense rollout with sparse supervision*. During the replay phase, the surrogate model still autoregressively unrolls predictions at the original, high-resolution Δt to physically propagate the temporal dynamics without stability loss. However, the replay loss is computed *only* at the sparse, stored target indices ($\mathcal{K}$). The insight is that while the operator requires dense intermediate steps to maintain physical stability, the pushforward supervision can safely be sparse due to the temporal spectral redundancy established in Section 4.2.

Instead of storing dense full-length pushforward windows, *Temporal Slicing* extracts a sampled context window along with a sliced target rollout. Let $\gamma \geq 1$ be the slice factor and $\mathcal{M}$ be the selection strategy. We define the set of sliced target indices within the pushforward horizon L_{pf} as:

$$\mathcal{K}(\gamma, \mathcal{M}) \subset \{1, 2, \ldots, L_{\text{pf}}\}. \tag{18}$$

The number of retained target steps in the rollout is:

$$m_\gamma = \left\lfloor \frac{L_{\text{pf}}}{\gamma} \right\rfloor. \tag{19}$$

For a selected training anchor t, the *sliced replay item* σ_t stores the intact L_{hist} context steps and only the m_γ target steps dictated by the slicing strategy $\mathcal{K}$:

$$\sigma_t = \left(\mathbf{u}_{t-L_{\text{hist}}+1:t},\ \{\mathbf{u}_{t+k}\}_{k\in\mathcal{K}(\gamma,\mathcal{M})}\right). \tag{20}$$

During the replay phase, the surrogate model still autoregressively unrolls predictions for the full L_{pf} horizon to physically propagate the temporal dynamics, but the replay loss is computed *only* at the stored sliced indices:

$$\mathcal{L}_{\text{TS}}(\theta; \mathcal{B}) = \mathbb{E}_{\sigma_t\sim\mathcal{B}}\left[\frac{1}{m_\gamma}\sum_{k\in\mathcal{K}} \ell\left(\hat{\mathbf{u}}_{t+k}, \mathbf{u}_{t+k}\right)\right], \tag{21}$$

where $\hat{\mathbf{u}}_{t+k} = f_\theta(\tilde{\mathbf{u}}_{t+k-L_{\text{hist}}:t+k-1})$, with $\tilde{\mathbf{u}}$ representing the autoregressive predictions, and ℓ is the step-wise prediction loss.

The memory advantage of Temporal Slicing comes from treating the input context and the target rollout differently. While a standard replay item must store all $L_{\text{hist}} + L_{\text{pf}}$ snapshots, Temporal Slicing retains the intact L_{hist} context steps—which are necessary to initialize the autoregressive model—and compresses only the dense rollout into m_γ sparse target steps. Consequently, the exact memory footprint (in terms of stored state snapshots) of a single sliced replay item is reduced to:

$$C_{\text{TS}} = L_{\text{hist}} + m_\gamma = L_{\text{hist}} + \left\lfloor \frac{L_{\text{pf}}}{\gamma} \right\rfloor. \tag{22}$$

In practice, if multiple sampled replay items overlap in time, they can share snapshots, meaning the actual number of unique stored snapshots can be even smaller than the sum of individual C_{TS} costs. Throughout our experiments, memory budgets are strictly compared using the absolute number of unique stored state snapshots rather than the raw count of replay items. This ensures that standard Replay and Replay-TS are evaluated fairly under the exact same snapshot capacity S_{total}.

Slicing mode While the slice factor γ controls the degree of compression in the target rollout, the choice of *which* target steps to retain affects how well the replay buffer preserves the governing dynamics. We consider four slicing modes for constructing the target index set $\mathcal{K}(\gamma, \mathcal{M})$:

- **Uniform Slicing:** Target indices are selected at evenly spaced locations across the pushforward horizon.
- **Stochastic Slicing:** Target indices are sampled uniformly at random from the pushforward horizon.
- **Initial Phase Slicing:** Target indices are selected from the earliest prediction steps of the pushforward horizon.
- **Terminal Phase Slicing:** Target indices are selected from the final portion of the pushforward horizon.

A visualization of different slicing modes is shown in Fig. 5. Furthermore, a schematic overview of our proposed method is shown in Fig. 1.

4.4 Empirical validation

To empirically validate the effectiveness of Temporal Slicing, we conduct two focused small experiments to analyze the slicing mode and the slice factor.

4.4.1 Impact of slicing mode

First, we investigate which sub-sequence selection strategy best preserves the learned operator. We design a continual learning scenario consisting of two distinct experiences using

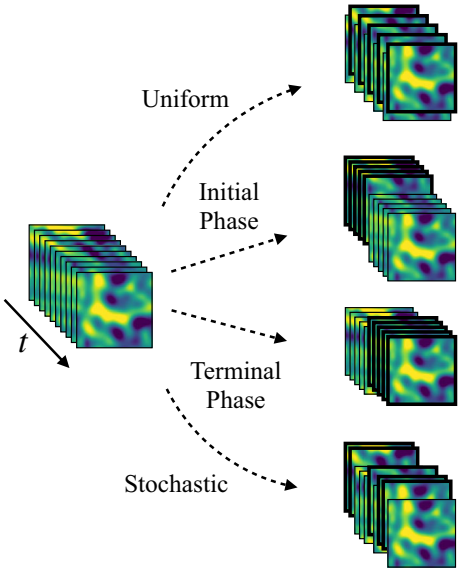


Fig. 5 Illustration of temporal slicing modes. Visual representation of how sub-sequences are extracted from a full continuous trajectory under Uniform, Stochastic, Initial Phase, and Terminal Phase strategies

the 1D Advection-Diffusion equation. To simulate a strict setting, we constrain the memory budget to a maximum snapshot capacity of $S_{\text{total}} = 500$. Since a single full simulation trajectory in this experiment contains $K_\tau = 100$ stored snapshots, the number of full trajectories that can be stored with this memory budget is equal to 5.

By applying Slice Factors of $\gamma = 2$ or $\gamma = 10$, we retain only a sub-sampled target sequence from the $L_{\text{pf}} = 50$ rollout steps (alongside the fixed $L_{\text{hist}} = 10$ context). For instance, $\gamma = 10$ means a replay item stores only $10 + (50/10) = 15$ snapshots, compared to 60 snapshots for an uncompressed context+rollout window or 100 for a full trajectory. Under the same fixed snapshot-capacity accounting ($S_{\text{total}} = 500$), this footprint reduction allows the replay buffer to retain information from a vastly larger number of distinct physical regimes. The results, evaluated as the average MSE loss on the test sets of both experiences at the end of training, are detailed in Table 1.

Table 1 Impact of slicing mode. All slicing modes are evaluated under the same fixed snapshot capacity ($S_{\text{total}} = 500$). Increasing the slice factor (γ) allows the buffer to retain replay windows from more trajectories under this capacity. Uniform and Stochastic slicing preserve the operator best, while Terminal Phase slicing fails to prevent catastrophic forgetting. They indicate the lowest value (lower error/loss) in each column, in all cases

Slice Mode	Average Stream Loss	
	$\gamma = 2$	$\gamma = 10$
Initial Phase	0.0030 ± 0.0005	0.0062 ± 0.0004
Terminal Phase	0.0357 ± 0.0027	0.2551 ± 0.0204
Stochastic	0.0029 ± 0.0008	0.0019 ± 0.0003
Uniform	$\mathbf{0.0027 \pm 0.0008}$	$\mathbf{0.0015 \pm 0.0003}$

The results reveal a stark contrast in performance based on the storage strategy. The *Terminal Phase* strategy results in catastrophic failure, as omitting the early evolutionary steps deprives the autoregressive model of the context needed to accurately map the initial condition $\mathbf{u}_0$. Conversely, *Uniform* and *Stochastic* strategies perform the best. We exclusively utilize the **Uniform** slice mode for all subsequent experiments because evenly covering the temporal dimension ensures the model captures both the initial transient dynamics and the long-term, quasi-steady-state evolution, preventing any bias toward early or late physical behaviors.

4.4.2 Effect of slice factor under fixed snapshot capacities

Next, we evaluate how the degree of compression (changing the Slice Factor γ) affects the surrogate model's ability to retain knowledge across a longer, more diverse data stream. For this experiment, the model is trained sequentially on 5 distinct physical regimes in the *1D Advection-Diffusion* equation. We evaluate the average stream loss across varying buffer constraints, defined by the absolute Snapshot Capacity (S_{total}). Given that each full trajectory contains $K_\tau = 100$ snapshots, a snapshot capacity of $S_{\text{total}} = 500$ corresponds to at most 5 full trajectories under standard replay. For Temporal Slicing, capacity is calculated using the exact number of stored states per sampled item ($C_{\text{TS}} = L_{\text{hist}} + L_{\text{pf}}/\gamma$). For the 1D experiments ($L_{\text{hist}} = 10$, $L_{\text{pf}} = 50$), a standard unsliced replay item ($\gamma = 1$) consumes 60 snapshots, while an aggressively sliced replay item ($\gamma = 25$) consumes only $10 + 2 = 12$ snapshots.

As shown in Table 2, we observe two distinct operational regimes:

1. **Budget-Constrained Setting ($S_{\text{total}} = 500$ or $S_{\text{total}} = 1000$):** When the memory footprint is highly restricted, storing full unsliced target rollouts ($\gamma = 1$) yields high error (0.0495), as the buffer hits capacity quickly and is forced to overwrite older physical regimes to accommodate new ones, causing physical regime collapse. Increasing the Slice Factor to 10 or 25 effectively multiplies the number of replay items, dropping the average stream loss (≈ 0.0328).
2. **Saturation Setting ($S_{\text{total}} = 20000$):** When the budget is large enough to natively store 200 full trajectories, the buffer inherently captures sufficient distribution diversity. In this scenario, aggressive slicing provides diminishing returns, and retaining longer, continuous temporal contexts ($\gamma = 1$ or 2) becomes marginally optimal.

These results demonstrate that Temporal Slicing acts as an effective mechanism for deploying PDE surrogates that deal with memory-constrained CL environments.

Table 2 Impact of slice factor on continual learning performance. Evaluated across a stream of 5 physical regimes. The Snapshot Capacity represents the absolute maximum number of time-steps the memory buffer can hold. For tight memory budgets (e.g., $S_{\text{total}} = 500$ or 1000), increasing the slice factor reduces the average stream loss by allowing the buffer to retain replay information from a broader set of trajectories and physical regimes. For unconstrained buffers ($S_{\text{total}} = 20000$), the benefit saturates. They indicate the lowest value (lower error/loss) in each column, in all cases

Slice Factor (γ)	Avg. Stream Loss $S_{\text{total}} = 500$	Avg. Stream Loss $S_{\text{total}} = 1000$	Avg. Stream Loss $S_{\text{total}} = 2000$	Avg. Stream Loss $S_{\text{total}} = 20000$
1 (Full Traj.)	0.0495 ± 0.0030	0.0422 ± 0.0050	0.0340 ± 0.0007	0.0277 ± 0.0005
2	0.0422 ± 0.0050	0.0340 ± 0.0007	0.0321 ± 0.0021	**0.0273 ± 0.0003**
10	**0.0328 ± 0.0005**	**0.0296 ± 0.0005**	**0.0289 ± 0.0006**	0.0290 ± 0.0009
25	**0.0328 ± 0.0004**	0.0328 ± 0.0013	0.0320 ± 0.0006	0.0319 ± 0.0011

4.5 Sensitivity analysis: autoregressive context and pushforward horizons

To evaluate the robustness of Temporal Slicing, we analyze its sensitivity to two critical autoregressive hyperparameters: the pushforward rollout length (L_{pf}) and the context history length (L_{hist}).

First, we investigate the impact of the target horizon during training by varying $L_{\text{pf}} \in \{10, 30, 50\}$ under a constrained memory budget. As shown in Fig. 6, standard full-trajectory replay ($\gamma = 1$) exhibits high variance and high mean error when the rollout horizon is short ($L_{\text{pf}} = 10$) under the restricted memory budget, only beginning to stabilize as the rollout extends to 50 steps. However, longer rollouts inherently exacerbate the memory bottleneck. Temporal Slicing ($\gamma = 2$) bypasses this tradeoff. It yields uniformly lower and highly stable stream losses (≈ 0.041) across all tested rollout lengths.

A potential counter-hypothesis is that standard replay's failure under strict memory constraints might be resolved simply by feeding the model a longer continuous input history, thereby enriching the local temporal dynamics and reducing the need for diverse samples. To test this, we evaluate the average stream loss across context lengths $L_{\text{hist}} \in \{10, 20, 30, 40\}$. The results (Fig. 7) show that expanding the context history significantly improves absolute predictive accuracy for both methods. This confirms that richer historical contexts aid the baseline operator in capturing temporal derivatives. However, the performance gap between the methods remains rigidly consistent. Temporal Slicing outpaces standard replay at every scale. This shows that while a longer history improves baseline fidelity, it cannot fully compensate for the catastrophic forgetting caused by physical regime collapse.

4.6 Physical regime diversity in the buffer

A core premise of *Temporal Slicing* is that compressing individual trajectories frees up capacity, allowing the replay buffer to accommodate a broader spectrum of physical regimes. To visualize how this expanded effective capacity translates into actual regime diversity, we map the retention distribution using a 1D Advection-Diffusion stream (comprising five sequential experiences). To isolate the effect of the memory bottleneck from the learning dynamics, we employ standard *reservoir sampling*, which provides a mathematically unbiased mechanism for retaining streaming data.

Figure 8 illustrates the normalized frequency of samples from each of the five physical regimes retained at the end of the data stream. Under strict memory constraints without slicing ($\gamma = 1$), the buffer suffers from severe *regime collapse*. Because each full trajectory consumes a massive share

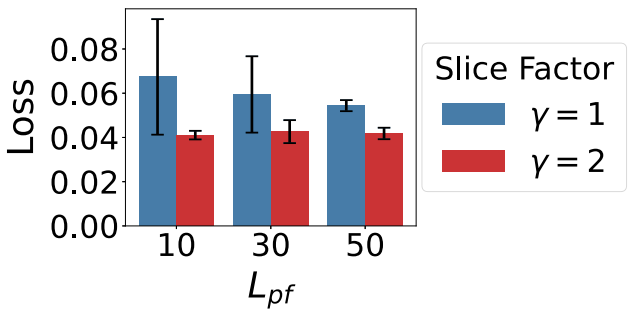


Fig. 6 Impact of pushforward rollout length (L_{pf}). Comparison of average stream loss for standard replay ($\gamma = 1$) and Temporal Slicing ($\gamma = 2$). Temporal Slicing maintains lower, stable error across all target sequence lengths, mitigating the high variance of standard replay at short horizons

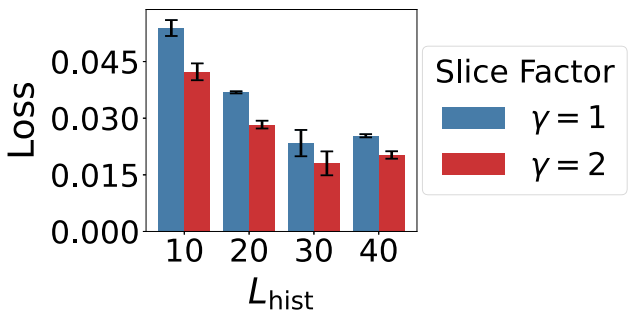


Fig. 7 Impact of context history length (L_{hist}). Expanding context history improves baseline operator fidelity, but Temporal Slicing ($\gamma = 2$) consistently outperforms standard replay ($\gamma = 1$) at every scale. This demonstrates that richer historical context cannot compensate for poor physical regime diversity

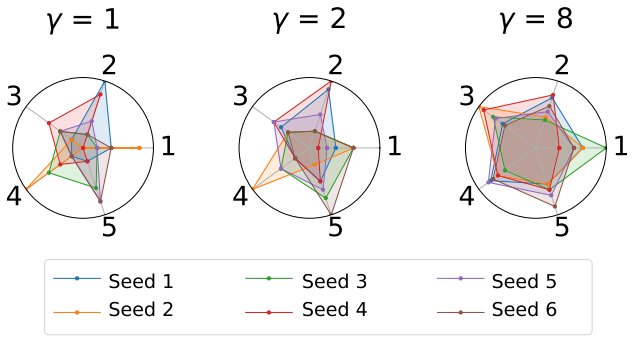


Fig. 8 Normal distribution of retained samples across five physical regimes (axes 1–5) using reservoir sampling for the item sampling mechanism. Without slicing ($\gamma = 1$), the buffer suffers from extreme variance and severe regime collapse (data starvation). Aggressive Temporal Slicing ($\gamma = 8$) significantly increases effective memory capacity, guaranteeing stable, symmetric retention across all encountered physics

of the memory budget, the buffer exhibits extreme variance across different random seeds. Consequently, older physical regimes are frequently overwritten entirely, manifesting as collapsed axes on the radar charts. This data starvation mathematically guarantees catastrophic forgetting. Conversely, as Temporal Slicing increases the effective buffer capacity ($\gamma = 2$ and $\gamma = 8$), the retention profile expands into a more stable, symmetric polygon, showing that the model continuously receives diverse supervisory signals from all previously encountered physical dynamics.

5 Experimental setup

To evaluate the plasticity-stability trade-off in learning simulation operators in a continual learning setting, we design a comprehensive benchmarking suite. Our experiments are structured to evaluate popular CL methods and our proposed *Temporal Slicing* method (Replay-TS) in maintaining high-fidelity predictions across sequentially evolving physical regimes, spanning from simple linear transport to chaotic fluid dynamics.

5.1 Underlying simulation datasets

We generate synthetic data for three time-dependent PDE families representing an increasing hierarchy of physical complexity. To accurately capture their distinct dynamics, we use tailored high-fidelity solvers: 1D/2D Advection-Diffusion and Burgers' equations are integrated using a Finite Volume Method (FVM). The 2D Navier-Stokes equations are solved via a pseudo-spectral method. All domains are periodic. Unless otherwise specified, initial conditions are sampled from a Gaussian Random Field (GRF) with correlation length $\ell = 0.08$ to ensure spectral diversity:

- **Advection-Diffusion (1D & 2D):** A fundamental linear PDE describing the transport and dispersion of a scalar field u. By varying the advection velocity a and the diffusion coefficient ν, we create experiences that differ in transport speed and smoothing rates. This tests the model's ability to adapt to linear spectral shifts.
- **Burgers' Equation (1D & 2D):** A canonical non-linear PDE often used as a simplified model for fluid mechanics. It combines convective non-linearity with diffusion, which leads to the formation of shock waves. We vary the viscosity parameter ν, which drastically alters the shock formation behavior.
- **Navier-Stokes Equation (2D):** The governing equations for viscous incompressible fluid flow. We utilize the vorticity-stream function formulation on the periodic unit torus. The complexity of the dynamics is controlled by the kinematic viscosity ν. By varying ν (which inversely determines the effective Reynolds number, $Re \propto 1/\nu$), we generate distinct physical regimes ranging from highly non-linear, chaotic flows at low viscosities to stable, dissipative flows at high viscosities, representing the most challenging test for operator generalization.

5.2 Data streams

To simulate non-stationary physical environments, we construct sequential learning streams $\mathcal{S} = \{\mathcal{E}_1, \ldots, \mathcal{E}_M\}$. For each PDE family, we generate datasets corresponding to 5 or 6 distinct parameter configurations (physical regimes). The first physical regime is reserved for *pre-training* to initialize the surrogate's representation, while the remaining 5 regimes constitute the continual learning stream.

We investigate two distinct stream types corresponding to the definitions in Section 3.2:

1. **Single-Physics Streams (Inter-Regime Shift):** The governing equation structure remains constant (e.g., Burgers'), but the physical coefficients $\boldsymbol{\mu}$ shift sequentially. This tests the model's robustness to parametric shifts within the same physical framework.
2. **Mixed-Physics Streams (Fundamental Equation Shift):** A more challenging scenario where the underlying governing laws themselves alternate. We construct:
   - **1D Mixed:** Alternating experiences between Advection-Diffusion and Burgers' dynamics.
   - **2D Mixed:** Alternating experiences between Advection-Diffusion and Navier-Stokes flows.

   In all mixed streams, the surrogate is trained on a common state representation within each dimensional setting. For the 1D mixed stream, both Advection-Diffusion and Burgers' samples are represented as scalar fields on the same one-dimensional grid. For the 2D mixed stream, Advection-Diffusion samples and Navier-Stokes vortic-

ity fields are represented as scalar fields on the same two-dimensional grid. Thus, the mixed-physics setting changes the underlying governing dynamics while preserving a compatible input-output representation for the surrogate.
These streams test the surrogate's ability to fundamentally reconfigure its internal operator approximation without overwriting the features required for the alternate physics. We list more details in Appendix A.

Finally, while the main benchmarking suite focuses on the parametric and structural shifts described above, we provide a supplementary analysis of **Intra-Regime Shifts** (robustness to distributional changes in initial conditions $\mathbf{u}_0$) in Appendix E.

5.3 Model architectures

To demonstrate that our method architecture-agnostic, we evaluate performance across two fundamentally different backbone architectures widely used in scientific machine learning:

- **Fourier Neural Operator (FNO):** A resolution-invariant architecture that learns the operator in the frequency domain. FNOs are global operators, making them theoretically more robust to discretization but potentially sensitive to spectral shifts caused by changing physics.
- **U-Net (Convolutional):** A standard encoder-decoder architecture operating in the spatial domain. The U-Net relies on local convolutions to capture spatial dependencies and is highly effective at capturing fine-scale features.

For more details, refer to Appendix C.

5.4 Training and pushforward hyperparameters

Across all experiments, we employ the pushforward trick to stabilize the autoregressive rollouts during training. The length of the historical context window is consistently set to $L_{\text{hist}} = 10$ time steps for both 1D and 2D experiments. The pushforward rollout horizon L_{pf} (the number of steps predicted into the future during a single training iteration) is configured based on the dimensionality of the problem:

- **1D Experiments:** The pushforward horizon is set to $L_{\text{pf}} = 50$ steps. Thus, a replay sample consists of 10 context steps and 50 target steps.
- **2D Experiments:** The pushforward horizon is set to $L_{\text{pf}} = 30$ steps. A replay sample consists of 10 context steps and 30 target steps.

These values are kept constant across all models and CL baselines to ensure a fair comparison of memory budgets and predictive capabilities.

5.5 Baselines

We benchmark our method **Replay-TS** against a representative suite of Continual Learning methods, as well as an upper-bound reference:

- **Joint Training (Oracle Reference):** The model is trained simultaneously on aggregated data from all experiences. This serves as an empirical oracle reference for performance when all regimes are available at once and no sequential forgetting occurs. It is not a formal upper bound, but it provides a useful target for evaluating the performance gap introduced by continual learning.
- **Naive (Fine-tuning):** The lower bound, where the model is trained sequentially with no mechanism to prevent forgetting.
- **EWC (Elastic Weight Consolidation):** A regularization-based approach that constrains the optimization trajectory by penalizing changes to weights critical for previous experiences.
- **Standard Experience Replay (ER):** The direct competitor, which stores full-length simulation trajectories. To ensure a fair comparison regarding memory footprint, all replay-based methods (including GEM, A-GEM, and our proposed Replay-TS) are strictly constrained to a fixed memory budget of $S_{\text{total}} = 2000$ snapshots.
- **GEM & A-GEM:** Gradient Projection methods (Gradient Episodic Memory and its Averaged variant) that utilize buffered samples to project gradients, ensuring they do not increase the loss on previous experiences.

5.6 Evaluation protocol

Experience-ID-aware, parameter-agnostic setting

We adopt an *experience-ID-aware* continual learning setting. At training and evaluation time, the surrogate receives a discrete experience identifier indicating which regime is being evaluated. This identifier is embedded as a low-dimensional vector and concatenated to the input state history. However, the model is not given the underlying physical coefficients $\boldsymbol{\mu}$, such as the diffusivity, viscosity, or transport velocity. In the mixed-physics streams, the model is also not given explicit numerical values describing the governing equation. It must learn the corresponding dynamics from the observed state histories, with the experience identifier serving only as an experience context signal.

Metrics

To ensure statistical significance, all main experiments are averaged over five independent runs with different random

Table 3 Quantitative comparison on 1D Single-Physics streams (FNO backbone). The *Joint* method serves as the upper bound for performance. All replay-based methods (Replay, GEM, A-GEM, Replay-TS) are constrained to a fixed snapshot capacity of $S_{\text{total}} = 2000$. The proposed Temporal Slicing (*Replay-TS*) consistently achieves the lowest error among sequential methods across both Advection-Diffusion and Burgers' streams. They indicate the lowest value (lower error/loss) in each column, in all cases

Method	1D Advection-Diffusion		1D Burgers'	
	MSE	Rel-L2	MSE	Rel-L2
Joint	0.0019 ± 0.0011	0.0094 ± 0.0019	0.0064 ± 0.0011	0.0187 ± 0.0079
Naive	2.5965 ± 0.0072	0.3454 ± 0.0017	1.5781 ± 0.0117	0.2623 ± 0.0009
EWC	2.1356 ± 0.0629	0.3196 ± 0.0037	2.1485 ± 0.1124	0.2793 ± 0.0068
Replay	0.0374 ± 0.0021	0.0469 ± 0.0016	0.0086 ± 0.0003	0.0248 ± 0.0006
GEM	0.0519 ± 0.0009	0.0503 ± 0.0030	0.0196 ± 0.0021	0.0380 ± 0.0017
A-GEM	0.0748 ± 0.0043	0.0628 ± 0.0029	0.1286 ± 0.0177	0.0964 ± 0.0075
Replay-TS	**0.0303** ± **0.0014**	**0.0414** ± **0.0004**	**0.0077** ± **0.0005**	**0.0232** ± **0.0005**

seeds governing stream generation and model weight initialization. We report:

1. **Average MSE:** The standard Mean Squared Error across all seen experiences at the end of the stream.
2. **Relative L_2 Error:** A normalized metric crucial for physics, as the magnitude of state variables can vary drastically between regimes (e.g., high-velocity vs. low-velocity flows).

6 Results

In this section, we present a comparative analysis of the proposed Temporal Slicing strategy (denoted as *Replay-TS*) against standard Continual Learning baselines. We focus primarily on the performance of the (standard) Fourier Neural Operator (FNO) backbone due to its prevalence in spectral PDE modeling. Some complementary results utilizing the U-Net architecture are detailed in Appendix G, demonstrating that our findings hold across different inductive biases.

6.1 1D single-physics streams

We begin by analyzing the 1D Advection-Diffusion and 1D Burgers' streams, which test the model's ability to adapt to changes in transport velocity and shock formation rates. Quantitative results are summarized in Table 3.

The *Joint* training baseline establishes the optimal achievable performance (e.g., an MSE of 0.0019 for Advection-Diffusion), demonstrating the model's inherent capacity when all data is simultaneously available. Conversely, the *Naive* baseline exhibits high error rates across both streams, confirming that catastrophic forgetting is immediate and severe; the specialized spectral features learned for earlier physical regimes are rapidly overwritten. Regularization-based methods (*EWC*) provide negligible improvement over the naive approach, suggesting that weight-stiffening alone cannot preserve the stability-plasticity trade-off for dynamic operators.

Standard Replay-based methods (*Replay*, GEM, A-GEM) successfully bridge this gap, reducing the MSE by two orders of magnitude compared to the Naive baseline. However, our proposed *Replay-TS* consistently outperforms standard Replay. In the Advection-Diffusion case, we observe a relative error reduction of approximately 18% compared to standard Replay.

To visually verify this, Fig. 9 presents a random sample from the test stream for the 1D Advection-Diffusion stream. While Replay-based methods reconstruct the general transport dynamics, closer inspection of the final time step ($t = 100$, bottom row) reveals that standard Replay exhibits noticeable phase misalignment and amplitude damping. In contrast, *Replay-TS* captures these fine-scale details with

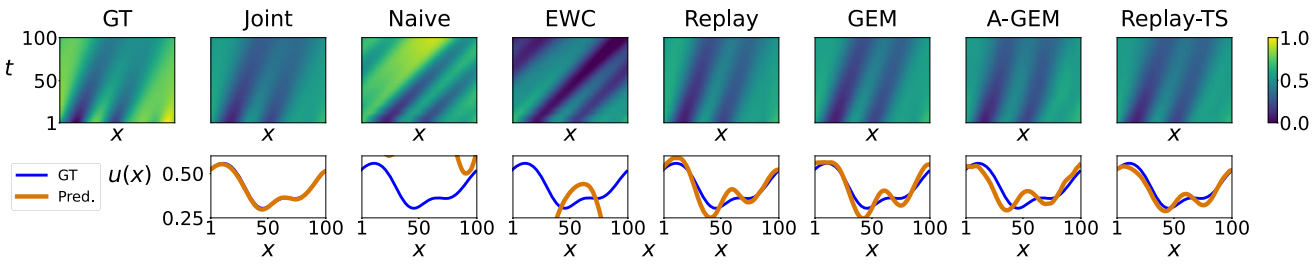


Fig. 9 Qualitative evaluation on the 1D advection-diffusion stream. The **Top Row** displays the spatiotemporal evolution of the scalar field. While Replay-based methods (Replay, GEM, A-GEM, Replay-TS) appear visually similar in the heatmap, the **Bottom Row** (snapshot at the final stored time step) reveals that standard Replay suffers from slight phase lag compared to the Ground Truth (red line), whereas Replay-TS (green line) aligns better

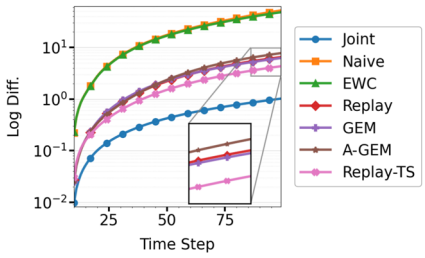


Fig. 10 Error propagation over autoregressive rollout for 1D advection-diffusion. Evolution of the logarithmic difference (Log Diff.) between model predictions and ground truth across the simulation time steps for the 1D Advection-Diffusion stream. While baseline and standard replay methods suffer from rapid exponential error compounding as the rollout progresses, *Replay-TS* maintains the lowest and most stable error profile over time among the CL methods, considerably mitigating the phase misalignment observed in standard ER

higher accuracy. This stability is further quantified in Fig. 10, which illustrates the logarithmic error over the autoregressive rollout. *Replay-TS* bounds the exponential error growth more effectively than standard Replay.

We observe a similar, albeit more modest, trend in the 1D Burgers' stream (Table 3). While the absolute MSE improvement of *Replay-TS* over standard Replay is incremental, the long-term rollout error plotted in Appendix Figs. 18 and 17 show that *Replay-TS* maintains a more stable error profile over time, preventing the late-stage divergence seen in standard Replay.

6.2 2D single-physics streams

The 2D experiments introduce higher spatial complexity. The Navier-Stokes sequences, in particular, require the model to retain knowledge of complex fluid behaviors across varying viscosity values under strict memory constraints. Results are presented in Table 4.

In these high-dimensional settings, the memory bottleneck of standard ER becomes critical. Full-trajectory replay fills the buffer quickly, severely limiting the diversity of past

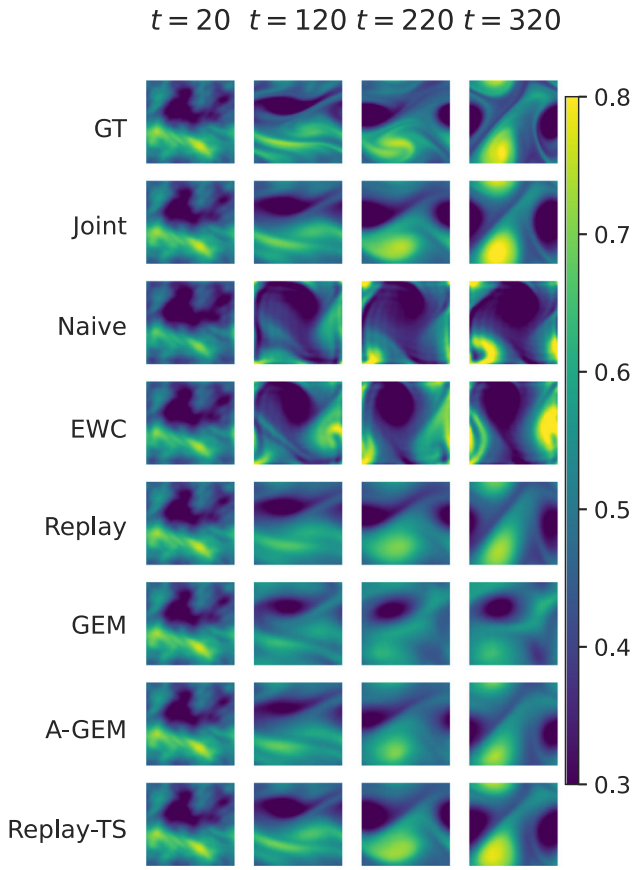


Fig. 11 Vorticity field evolution on the 2D Navier-Stokes stream. Comparison of (from top to bottom) Ground Truth, Naive, Replay, and Replay-TS predictions across time steps $t \in \{20, 120, 220, 320\}$

viscosity regimes that can be stored. By slicing trajectories, *Replay-TS* maintains a higher effective physical regime diversity. This is reflected in the Navier-Stokes stream, where *Replay-TS* achieves an MSE of 0.1005 compared to 0.1464 for standard Replay. Interestingly, gradient projection methods (GEM, A-GEM) struggle here, often underperforming simple Replay, likely due to the difficulty of computing reliable projection constraints on high-variance fluid data.

Table 4 Evaluation on 2D high-dimensional streams (FNO backbone). *Joint* training indicates the optimal achievable performance. Under a fixed memory budget of $S_{\text{total}} = 2000$, in memory-constrained settings like the Navier-Stokes stream, *Replay-TS* outperforms full-trajectory Replay (MSE 0.1005 vs 0.1464) by maximizing the diversity of physical regimes stored in the buffer. They indicate the lowest value (lower error/loss) in each column, in all cases

Method	2D Adv-Diff		2D Burgers'		2D Navier-Stokes	
	MSE	Rel-L2	MSE	Rel-L2	MSE	Rel-L2
Joint	0.0018 ± 0.0002	0.0136 ± 0.0004	0.0042 ± 0.0006	0.0245 ± 0.0003	0.0428 ± 0.0012	0.0054 ± 0.0021
Naive	0.1684 ± 0.0040	0.1275 ± 0.0014	0.8816 ± 0.0412	0.2081 ± 0.0074	1.4629 ± 0.1095	0.1860 ± 0.0103
EWC	0.1377 ± 0.0058	0.1214 ± 0.0023	0.7950 ± 0.0728	0.1986 ± 0.0112	1.4024 ± 0.1988	0.1809 ± 0.0279
Replay	0.0041 ± 0.0001	0.0210 ± 0.0001	0.0115 ± 0.0005	0.0324 ± 0.0008	0.1464 ± 0.0060	0.0137 ± 0.0008
GEM	0.0206 ± 0.0029	0.0533 ± 0.0046	0.0343 ± 0.0024	0.0594 ± 0.0035	0.2125 ± 0.0182	0.0194 ± 0.0130
A-GEM	0.0047 ± 0.0001	0.0246 ± 0.0003	0.0308 ± 0.0015	0.0508 ± 0.0022	0.1880 ± 0.0097	0.0172 ± 0.0010
Replay-TS	**0.0039** ± **0.0004**	**0.0188** ± **0.0002**	**0.0100** ± **0.0005**	**0.0297** ± **0.0012**	**0.1005** ± **0.0029**	**0.0096** ± **0.0009**

Table 5 Performance on Mixed-Physics streams (FNO backbone). This setting alternates between fundamentally different governing equations. Compared to the *Joint* training target, *Replay-TS* demonstrates a superior stability-plasticity trade-off under a snapshot capacity of $S_{\text{total}} = 2000$, reducing the MSE by over 30% in the mixed scenario compared to standard Replay. They indicate the lowest value (lower error/loss) in each column, in all cases

Method	1D Mixed		2D Mixed	
	MSE	Rel-L2	MSE	Rel-L2
Joint	0.0060 ± 0.0017	0.0476 ± 0.0008	0.1699 ± 0.0037	0.0155 ± 0.0021
Naive	0.6485 ± 0.0158	0.5522 ± 0.0124	7.2978 ± 0.1589	0.3061 ± 0.0046
EWC	0.6488 ± 0.0074	0.5786 ± 0.0113	6.4730 ± 0.2058	0.2834 ± 0.0090
Replay	0.0354 ± 0.0159	0.1239 ± 0.0057	1.4856 ± 0.1219	0.0615 ± 0.0041
GEM	0.0422 ± 0.0038	0.1574 ± 0.0062	2.1316 ± 0.0471	0.0868 ± 0.0045
A-GEM	0.0672 ± 0.0059	0.2128 ± 0.0066	1.7476 ± 0.0364	0.0682 ± 0.0025
Replay-TS	**0.0218** ± **0.0065**	**0.0831** ± **0.0032**	**0.9709** ± **0.0048**	**0.0456** ± **0.0004**

We analyze the qualitative behavior of the surrogate in the Navier-Stokes stream in Fig. 11. The visualizations reveal that while standard Replay maintains the global structure, it accumulates higher error and has a lower fidelity in comparison to the Joint method. *Replay-TS* effectively improves the prediction for the latter steps.

6.3 Mixed physics streams

Table 5 details the performance on the Mixed Physics streams, where the learner must alternate between fundamentally different governing equations (e.g., switching between linear Advection-Diffusion and non-linear Burgers' or Navier-Stokes). This represents the most challenging scenario.

While *Joint* training successfully learns the union of these alternating dynamics, sequential methods struggle to retain the mutually exclusive features required for different PDEs. Specifically, this setting is highly susceptible to "cross-contamination," where the convolutional or spectral filters learned for one physical regime inappropriately manifest in the predictions of the other. The *Replay-TS* strategy excels here, particularly in the 2D Mixed case, reducing the MSE by over 30% compared to standard replay (0.9709 vs 1.4856). This suggests that when the experience distribution is highly non-stationary (changing governing laws), maximizing the entropy of the replay buffer via slicing is highly effective in preventing the model from overwriting these distinct features and overfitting to the most recently seen physics.

We validate these findings qualitatively in Fig. 12 for the 1D Mixed stream. The *Naive* model exhibits severe catastrophic forgetting, predicting physically inconsistent states (bright yellow artifacts). Standard *Replay* recovers the general structure but leaves non-trivial residual errors (deep blue regions). In contrast, *Replay-TS* preserves the highest fidelity, with a largely uniform and low-error difference map among the CL methods. The spectral and spatial stability of *Replay-TS* is similarly maintained in the 2D Mixed stream, as visualized in Appendix Fig. 20.

7 Discussion and conclusion

The deployment of machine learning surrogates in scientific computing is dependent on their ability to reliably adapt to evolving physical parameter regimes. In this work, we formulated the adaptation of autoregressive PDE surrogates as a Continual Learning (CL) problem and systematically evaluated standard methods to mitigate catastrophic forgetting.

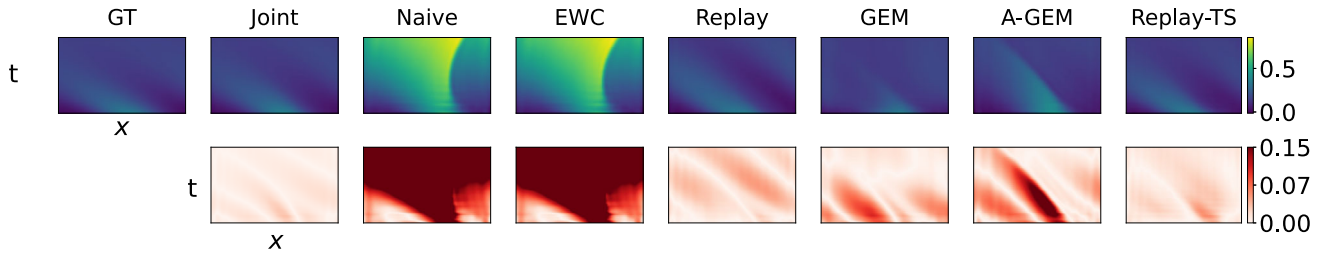


Fig. 12 Qualitative comparison on the 1D mixed physics stream. **Top Row** visualizes the spatiotemporal predictions and the bottom row shows the absolute difference between the prediction and the ground-truth. The *Naive* model suffers from catastrophic forgetting, failing to recall the correct transport dynamics and predicting a physically invalid state (visible as a bright yellow artifact). While standard *Replay* recovers the general structure, the difference map in the **Bottom Row** shows significant residual error (dark blue regions). *Replay-TS* achieves the highest fidelity, with the difference map remaining largely uniform and low-error, indicating robust retention of the alternating physical laws

7.1 The efficacy of standard experience replay

Our empirical analysis highlights that standard Experience Replay (ER) can be a robust and efficient baseline for continual operator learning given enough model capacity and buffer size. Despite the availability of more mathematically complex regularization (e.g., EWC) and gradient-projection techniques (e.g., GEM, A-GEM), simply rehearsing past trajectories often provides superior stability. This is particularly pronounced in physical systems, where the data manifold is smooth and governed by strict conservation laws, which allows simple replay to effectively anchor the neural network's parameters against catastrophic shifts. We suggest that for many standard scientific machine learning applications, standard Replay should be considered the primary baseline before adopting more computationally expensive CL frameworks.

7.2 Enhancing replay via temporal slicing

Despite the strong performance of standard ER, the high-dimensional nature of physics simulations exposes a severe memory bottleneck. Because single trajectories contain thousands of dense spatial snapshots, a strict memory budget forces standard Replay to store very few distinct physical regimes.

To address this, we introduced *Temporal Slicing* (*Replay-TS*). Crucially, unlike standard sequence subsampling—which coarsens the integration time-step and risks numerical instability—Replay-TS uniquely combines dense autoregressive unrolling with sparse temporal supervision. By recognizing the spectral redundancy in stable simulations, our method trades continuous temporal length for a wider coverage of the parameter space $\mathcal{P}$. While the quantitative reduction in average MSE may appear incremental in simpler, single-physics streams, we demonstrated that these gains are crucial in the context of autoregressive modeling. Because small local errors compound exponentially during long-horizon rollouts, the enhanced regime diversity provided by Temporal Slicing effectively bounds this error growth, preserving finer-scale structural details and high-frequency spectral energy that standard Replay gradually smooths out.

7.3 Limitations and future directions

While Temporal Slicing improves memory efficiency, it relies on the assumption that useful replay information can be retained through a sparse set of temporally distributed autoregressive windows. This assumption is well suited to the dissipative regimes studied here, where temporal spectra are dominated by low-frequency components. However, for systems exhibiting high-frequency dynamics, sparse slicing risks aliasing the underlying physics or entirely missing critical structural phenomena. In such extreme cases, the temporal redundancy assumption breaks down, and the method may require denser temporal storage or adaptive, frequency-aware slicing strategies.

Additionally, while Replay-TS is memory-efficient regarding raw snapshot storage, our current evaluation focuses on streams of up to five distinct regimes. Scaling this approach to deployments with dozens of continuously shifting physical regimes over long operational lifetimes will require further investigation into adaptive memory allocation and the management of minor indexing overheads required to handle highly fragmented buffer structures.

Furthermore, our proposed method operates in an experience-ID-aware setting, relying on an explicit experience identifier during training and evaluation. To move toward fully autonomous simulation digital twins, future systems should become experience-ID-agnostic and detect regime shifts without manual intervention. One possible direction is to use spikes in autoregressive prediction error as a *surprisal signal* for distribution shift detection. Such a mechanism could trigger buffer updates or model adaptation automatically when the surrogate encounters previously unseen physical dynamics.

In conclusion, *Temporal Slicing* offers a simple, architecture-agnostic enhancement to standard Experience Replay. By bridging the gap between static offline training and the dynamic memory constraints of real-time simulation, it represents a practical step toward adaptive scientific machine learning.

Appendix A: Dataset generation details

To create data streams, synthetic simulation datasets for the Advection-Diffusion (1D and 2D), Burgers' (1D and 2D) and Navier-Stokes (2D) equations were generated using pseudo-spectral methods (Crank-Nicolson for linear, Runge-Kutta for non-linear terms). All initial conditions (ICs) were sampled from a Gaussian Random Field (GRF) with correlation length $\ell = 0.08$ to ensure spectral diversity. The domain boundaries are periodic in all cases. The details for each equation are given in the subsequent sub-sections.

A.1: Advection-diffusion (1D & 2D)

The streams consist of 6 experiences generated by the Cartesian product of velocity and diffusivity sets (Table 6). For **1D:** $N_x = 100$, and in **2D:** 64×64. In both cases we use $T = 1.0$, $K_\tau = 100$.

Table 6 Advection-diffusion parameters

Param	1D Values	2D Values
Diff. (ν)	0.001, 0.004, 0.008	0.004, 0.007, 0.01
Vel. (a / $\mathbf{v}$)	0.3, 0.8	$(-0.1, 0.5)(0.1, 0.5)$

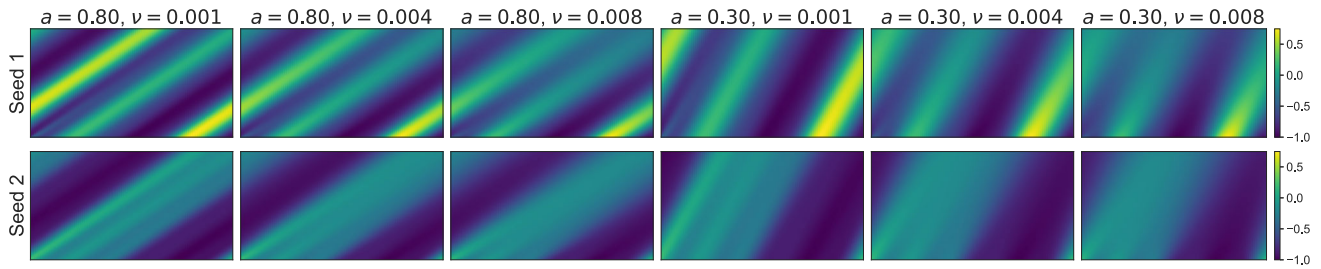


Fig. 13 Visualization of the 1D Advection-Diffusion stream samples. The rows represent independent simulation trajectories initialized with different random seeds (Initial Conditions). The columns correspond to spatiotemporal plots for distinct physical regimes characterized by varying advection velocity a and diffusivity ν, starting from the same initial condition. The heatmaps depict the full spatiotemporal evolution $u(x, t)$, where the x-axis represents spatial position and the y-axis represents time

Figure 13 illustrates the diversity of the generated data for the 1D case. The figure displays the spatiotemporal evolution of the scalar field for different parameter combinations. It highlights how variations in advection velocity a alter the slope of propagation (visible as the angle of the wavefronts in the heatmap), while diffusivity ν affects the smoothing rate of the wave profiles over time.

A.2: Burgers' equation (1D & 2D)

We control the Shock formation difficulty by viscosity ν as given in Table 7 to create distinct experiences. Lower ν yields sharper gradients. For **1D:** $N_x = 100$, and in **2D:** 64×64. Similar to the Advection-Diffusion equation, both use $T = 1.0$, $K_\tau = 100$.

Table 7 Burgers' viscosity regimes (ν)

Dim.	Viscosity Coefficients
1D	0.001, 0.003, 0.005, 0.008, 0.01
2D	0.001, 0.005, 0.01, 0.02, 0.03

Figure 14 provides a visual comparison of the 2D Burgers' regimes. The figure displays the state of the system at the final time step ($T = 1.0$) for two independent random initializations. As observed in the visualization, decreasing the viscosity ν (moving from right to left) leads to the formation of sharper shock fronts and more distinct, non-smooth structures, whereas higher viscosity regimes result in smoother, more dissipative fields.

A.3: Navier-Stokes (2D)

We simulate the vorticity evolution on a torus (64×64, $T = 5.0$). The stream consists of 6 experiences varying by kinematic viscosity ν, spanning turbulent to laminar flows (Table 8).

Table 8 Navier-Stokes viscosity regimes

Regime Type	Viscosity (ν)
Turbulent (Exps 1–2)	7×10^{-5}, 9×10^{-5}
Transitional (Exps 3–4)	2×10^{-4}, 4×10^{-4}
Laminar (Exps 5–6)	6×10^{-4}, 8×10^{-4}

A.4: 1D mixed stream (advection-diffusion ↔ Burgers')

This stream interleaves 1D linear transport with 1D nonlinear shock dynamics. The sequence consists of 8 experi-

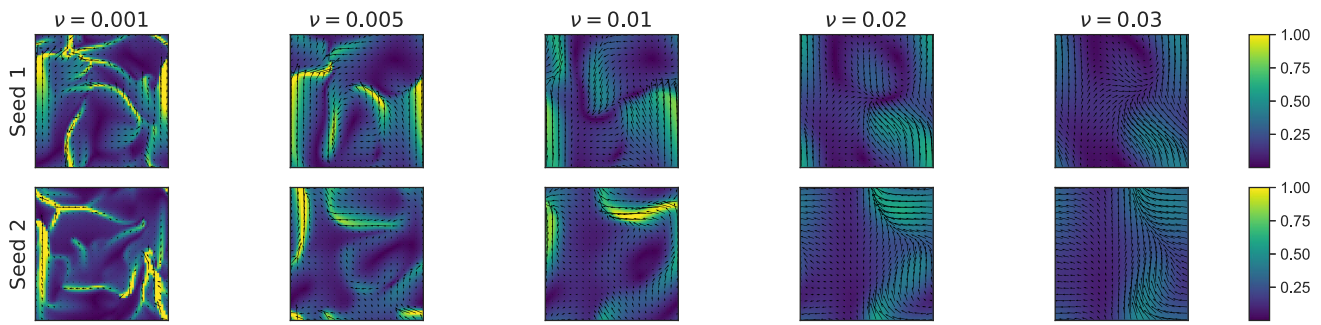


Fig. 14 Sample visualization for the 2D Burgers' equation. Each row correspond a random initial conditions (fixed random seed). The columns display the final snapshot of each physical regime (at $T = 1.0$) starting from the same initial condition (in the row). Each physical regime has a different viscosity coefficient ν. Note the transition from sharp, shock-dominated structures at low ν (left) to smooth, diffusive fields at high ν (right)

ences, alternating between Advection-Diffusion (a, ν) and Burgers' (ν) equations. In Table 9, we show the sequence of experiences with their corresponding physical parameters.

Table 9 1D mixed stream curriculum

Experience	Equation	Parameters
1	Adv-Diff	$\nu = 0.008, a = 0.8$
2	Burgers'	$\nu = 0.008$
3	Adv-Diff	$\nu = 0.008, a = 0.3$
4	Burgers'	$\nu = 0.005$
5	Adv-Diff	$\nu = 0.004, a = 0.8$
6	Burgers'	$\nu = 0.01$
7	Adv-Diff	$\nu = 0.004, a = 0.3$
8	Burgers'	$\nu = 0.001$ (Shock)

A.5: 2D mixed stream (advection-diffusion ↔ Navier-Stokes)

This stream alternates between 2D linear transport and chaotic fluid flow (Navier-Stokes). The Navier-Stokes experiences oscillate between laminar and turbulent regimes, while the Advection-Diffusion experiences vary the transport velocity field direction as shown in Table 10.

Table 10 2D mixed stream curriculum

Experience	Equation	Regime Details
1	Adv-Diff	$\nu = 0.004, \mathbf{v} = (-0.1, 0.5)$
2	Nav-Stokes	$\nu = 6 \cdot 10^{-4}$ (Laminar)
3	Adv-Diff	$\nu = 0.007, \mathbf{v} = (0.1, 0.5)$
4	Nav-Stokes	$\nu = 9 \cdot 10^{-5}$ (Turbulent)
5	Adv-Diff	$\nu = 0.010, \mathbf{v} = (-0.1, 0.5)$
6	Nav-Stokes	$\nu = 8 \cdot 10^{-4}$ (Laminar)
7	Adv-Diff	$\nu = 0.010, \mathbf{v} = (0.1, 0.5)$
8	Nav-Stokes	$\nu = 7 \cdot 10^{-5}$ (Turbulent)

Appendix B: Notation table

Table 11 summarizes the key mathematical symbols, their physical or algorithmic meanings, and the sections where they are primarily introduced.

Table 11 Summary of notations

Symbol	Description
Physical System & Discretization	
$\Omega, \mathcal{T}$	Spatial domain and temporal interval
$\mathbf{u}(\mathbf{x}, t)$	Continuous physical state field
$\mathbf{u}_t$	Discretized spatial state vector at time step t
$\boldsymbol{\mu} \in \mathcal{P}$	Vector of physical parameters (e.g., advection velocity a, viscosity ν)
$\mathcal{N}$	Non-linear spatial differential operator
$\Phi_{\Delta t}$	Ground-truth discrete evolution operator (numerical time-stepper)
τ	Discrete simulation trajectory
K_τ	Total number of snapshots (states) stored in a full trajectory
Surrogate Modeling & Training	
f_θ	Neural network surrogate parameterized by trainable weights θ
L_{hist}	Autoregressive context window length (number of history steps)
L_{pf}	Pushforward rollout horizon (target prediction steps)
Continual Learning & Experience Replay	
$\mathcal{S} = \{\mathcal{E}_i\}_{i=1}^M$	Sequential stream of M distinct experiences (physical regimes)
$\mathcal{D}_i^{\text{train}}, \mathcal{D}_i^{\text{test}}$	Training and evaluation datasets for experience $\mathcal{E}_i$
$\mathcal{B}$	Episodic memory (replay) buffer
S_{total}	Maximum snapshot capacity (memory budget) of the replay buffer
Temporal Slicing (Replay-TS)	
γ	Slice factor (degree of temporal compression)
$\mathcal{K}(\gamma, \mathcal{M})$	Set of retained target indices under slicing mode $\mathcal{M}$
m_γ	Number of retained rollout steps ($m_\gamma = \lfloor L_{\text{pf}}/\gamma \rfloor$)
σ_t	Sliced replay item anchored at time step t
C_{TS}	Memory footprint (number of snapshots) for a single sliced replay item

Appendix C: Model architectures

All surrogate models were implemented in PyTorch. In our experiments we employ two different types of architecture, namely Fourier Neural Operator (FNO) and U-Net. To implement the experience-ID-aware setting, each experience i is associated with a low-dimensional embedding $\mathbf{e}_i \in \mathbb{R}^2$. This embedding is spatially broadcast and concatenated to the input state history $\mathbf{u}_{t-L_{\text{hist}}+1:t}$ before the first network layer. The embedding identifies the experience index but does not encode the physical coefficients $\boldsymbol{\mu}$ directly. The details of each model architecture are given in the subsequent subsections.

C.1: Fourier Neural Operator (FNO)

The FNO approximates the solution operator in the frequency domain, offering resolution invariance. The architecture lifts the input to a high-dimensional channel space via a linear

projection P and processes it through four spectral convolution layers. The update rule for layer l is defined as:

$$v_{l+1}(\mathbf{x}) = \sigma\left(W v_l(\mathbf{x}) + \mathcal{F}^{-1}\left(R_\phi \cdot \mathcal{F}(v_l)\right)(\mathbf{x})\right) \tag{C1}$$

where $\mathcal{F}$ is the Fourier transform and R_ϕ is a complex-valued tensor that filters the lowest frequency modes.

- **Mode Truncation:** For 1D, we retain the lowest 16 modes. For 2D, we utilize a "corner-based" retention strategy, keeping the lowest 12×12 modes to capture dominant large-scale dynamics while discarding high-frequency noise.
- **Grid Features:** Deterministic coordinates are appended to the input to break translation invariance, essential for non-periodic boundary conditions.

C.2: U-Net

The U-Net serves as a strong spatial baseline, employing a symmetric encoder-decoder with skip connections to preserve high-frequency details.

- **Dynamic Padding:** To handle arbitrary spatial discretizations, the input is dynamically padded to be divisible by 2^{depth} (16). This prevents dimension mismatch errors during the concatenation of encoder features with upsampled decoder features.
- **Initialization:** We employ Kaiming initialization for internal layers but explicitly zero-initialize the final 1×1 projection layer. This ensures the model starts with neutral predictions, stabilizing the early phase of meta-learning.

C.3: Hyperparameter configuration

Tables 12 and 13 summarize the architecture hyperparameters used in the experiments. The embedding dimension is 2 in all cases, and embeddings are kept fixed.

Table 12 FNO architecture specifications. Hyperparameters used for the 1D and 2D Fourier Neural Operator backbones

Parameter	FNO-1D	FNO-2D
Output channels	1	1
Width	64	20
Fourier modes	16	12×12
Fourier blocks	4	4
Grid features	x	x, y
Experience embedding dim.	2	2

Table 13 U-Net architecture specifications. Hyperparameters used for the 1D and 2D U-Net backbones

Parameter	U-Net-1D	U-Net-2D
Output channels	1	1
Base channels	32	32
Downsampling levels	4	4
Grid features	x	x, y
Experience embedding dim.	2	2

Appendix D: CL methods

To benchmark the effectiveness of our proposed method, we compare it against a diverse suite of CL methods. These baselines were selected to represent the three dominant paradigms in the field: *Regularization*, *Replay*, and *Gradient Projection*. Below, we detail the mechanics of each approach and their specific implications for learning physical operators.

D.1: Joint training (the upper bound)

Joint Training is not a continual learning strategy, but rather the traditional offline learning paradigm used to establish the maximum achievable performance of the surrogate model.

- **Mechanism:** The model is trained on an aggregated dataset containing all experiences $\mathcal{D}_{\text{joint}} = \bigcup_{k=1}^{M} \mathcal{D}_k$ simultaneously. The data is shuffled across experiences, effectively removing any temporal distribution shifts.
- **The Baseline Value:** This provides an empirical upper bound for the model's capacity to learn the union of all physical operators without the confounding factor of catastrophic forgetting. Any gap between Joint Training and a CL method isolates the performance loss strictly due to sequential learning.

D.2: Naive fine-tuning (the lower bound)

The *Naive* approach represents the default behavior of a neural network when exposed to sequential data without any specific intervention. It serves as the experimental control group to quantify the severity of catastrophic forgetting.

- **Mechanism:** When the physical regime shifts from $\mathcal{E}_{k-1}$ to $\mathcal{E}_k$, the optimizer minimizes the loss solely on the current dataset $\mathcal{D}_k$, completely disregarding the parameter configuration θ^*_{k-1} learned previously.
- **Limitation:** In PDE modeling, this results in the immediate overwriting of spectral features. For example, if a model adapts to a high-diffusivity regime, it updates its convolutional filters to smooth out gradients. When tested on a previous low-diffusivity experience, these smoothed filters fail to capture shock waves, resulting in what we term "Catastrophic Smoothing."

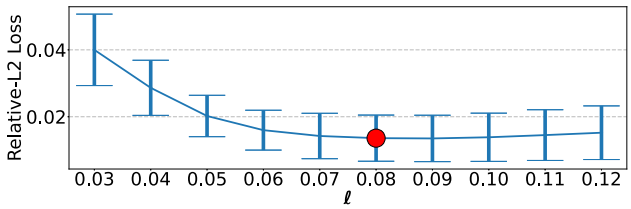


Fig. 15 Sensitivity to correlation length (ℓ). The average rollout Relative L2 error over 50 simulations for varying initial condition smoothness. The model, trained on $\ell = 0.08$, exhibits minimal error at the source regime, with performance degrading as the spectral properties of the initial condition shift

D.3: Elastic Weight Consolidation (EWC)

Elastic Weight Consolidation [52] attempts to mitigate forgetting without storing past data, opting instead to "stiffen" the weights of the neural network.

- **Mechanism:** EWC assumes that not all network parameters are equally important for a given physical experience. After training on experience $i-1$, it computes the diagonal of the Fisher Information Matrix F, where each diagonal entry F_k approximates the sensitivity of the loss to parameter θ_k. When training on the current experience i, a quadratic regularizer is added to the loss function to penalize deviations from the previous optimal parameters:

$$\mathcal{L}_{\text{EWC}}(\theta) = \mathcal{L}(\theta; \mathcal{D}_i^{\text{train}}) + \frac{\lambda}{2} \sum_k F_k (\theta_k - \theta^*_{i-1,k})^2 \quad \text{(D2)}$$

D.4: Standard Experience Replay (ER)

Experience Replay [59] is the "brute force" memory approach. It operates on the principle that the best way to remember the past is to constantly rehearse it.

- **Mechanism:** The model maintains a FIFO (First-In-First-Out) memory buffer $\mathcal{B}$ containing a random subset of raw data from previous experiences. During training, every batch of new physics data is concatenated with a batch sampled from $\mathcal{B}$. To establish a fair comparison with our proposed method, the capacity of this buffer is strictly fixed at $S_{\text{total}} = 2000$ snapshots across all replay-based baselines.
- **The Trade-off:** While highly effective, ER shifts the problem from an algorithmic challenge to a storage challenge. In our context, storing full simulation trajectories (e.g., 1000 timesteps of high-resolution fluid flow) is memory-prohibitive. ER forces a choice: store very few high-fidelity examples (risking overfitting) or store many low-fidelity examples (losing physics). This is the specific bottleneck our *Temporal Slicing* method addresses.

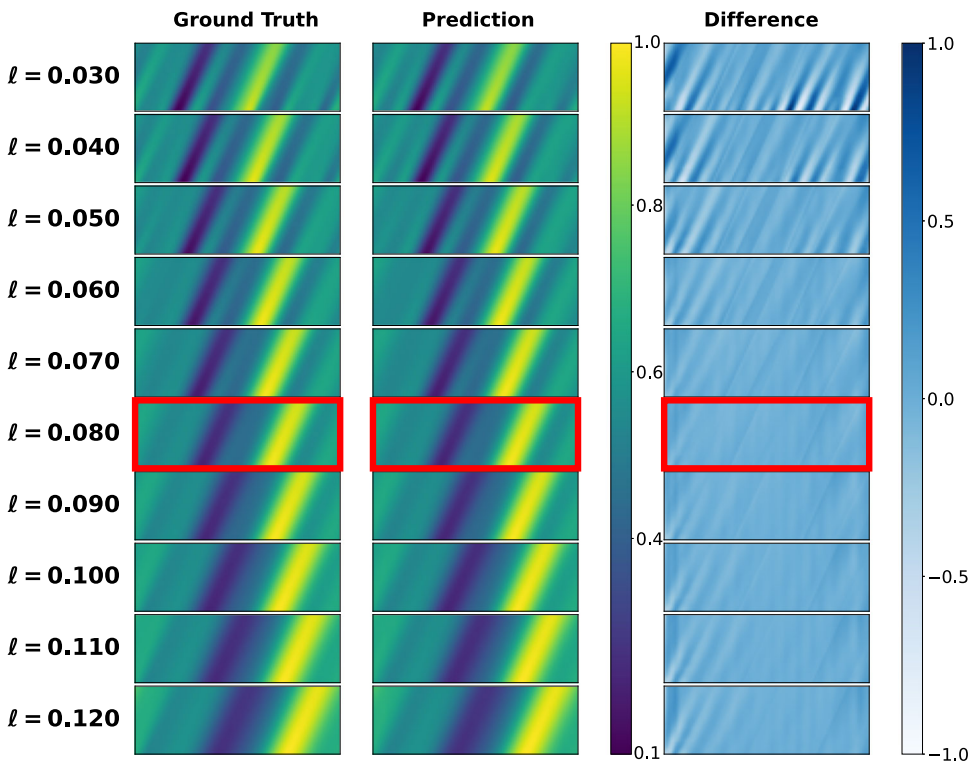


Fig. 16 Rollout visualization under distribution shift. Comparison of Ground Truth, Prediction, and Error Difference for different correlation lengths ℓ. The red box highlights the training regime ($\ell = 0.08$). Deviations from this regime result in increased error, demonstrating that the surrogate is not robust to distributional shifts in $\mathbf{u}_0$, even when the physics remains constant

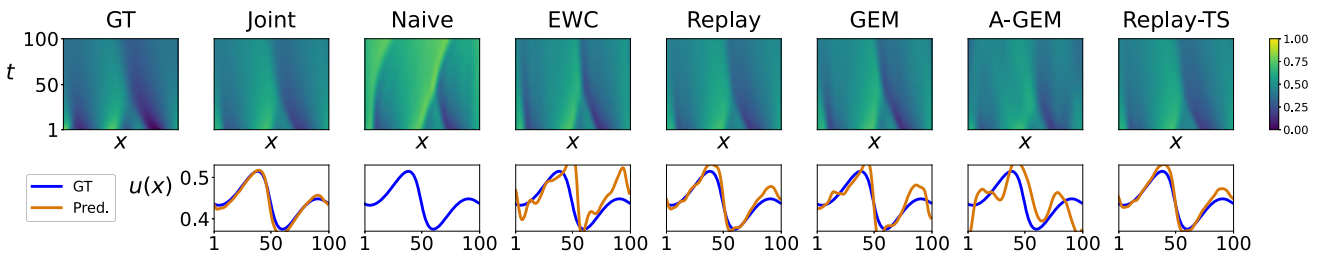


Fig. 17 Qualitative evaluation on the 1D Burgers stream. The spatiotemporal evolution of the scalar field comparing different CL methods. While Replay recovers the shock formation better than Naive and EWC, *Replay-TS* produces slightly smoother prediction what makes it more aligned with the Ground Truth

D.5: Gradient Episodic Memory (GEM)

GEM [62] transforms Continual Learning into a constrained optimization problem. Rather than just mixing data (like ER), it actively modifies the optimization trajectory.

- **Mechanism:** At every step, GEM calculates the gradient g on the current experience and the gradients g_k on samples from previous experiences stored in memory. It then checks for "alignment." If the update for the current experience would increase the error on a previous experience (indicated by an obtuse angle between gradients, $\langle g, g_k \rangle < 0$), GEM projects g to the closest vector $\tilde{g}$ that satisfies $\langle \tilde{g}, g_k \rangle \geq 0$.
- **The Computation Cost:** While mathematically elegant, GEM requires solving a Quadratic Programming (QP) problem at every iteration to find the projection. For high-dimensional PDE surrogates with millions of parameters, this computational overhead is often intractable, slowing down training by orders of magnitude.

D.6: Averaged-GEM (A-GEM)

A-GEM [63] is the streamlined successor to GEM, designed to balance the rigor of gradient projection with the speed required for deep learning.

- **Mechanism:** Instead of ensuring the gradient does not hurt *each* individual previous experience (which requires one constraint per experience), A-GEM ensures the gradient does not hurt the *average* performance of the memory buffer. It computes a single reference gradient g_{ref} from a batch of random buffer samples. If $\langle g, g_{\text{ref}} \rangle < 0$, the update is projected:

$$\tilde{g} = g - \frac{\langle g, g_{\text{ref}} \rangle}{\langle g_{\text{ref}}, g_{\text{ref}} \rangle} g_{\text{ref}} \tag{D3}$$

- **Comparison:** This relaxes the strict constraints of GEM, allowing for faster training. However, in mixed-physics streams (e.g., alternating between Advection and Burgers'), the "average" gradient might be physically meaningless, leading to suboptimal updates compared to pure Replay.

Appendix E: Sensitivity to initial condition distribution

While the primary focus of this work is on *Inter-Regime* shifts (parametric variations in $\boldsymbol{\mu}$), the robustness of data-driven surrogates against *Intra-Regime* shifts, variations in the distribution of initial conditions $P(\mathbf{u}_0)$, is also interesting and important. In this section, we empirically analyze the sensitivity of a pre-trained model to changes in the initial condition, even when the governing physical coefficients remain constant.

E.1: Experimental setup

We utilize the 1D Advection-Diffusion equation with fixed physical parameters ($a = 0.3$, $\nu = 0.001$). The initial conditions $\mathbf{u}_0(x)$ are sampled from a Gaussian Random Field (GRF) where the smoothness is controlled by the correlation length ℓ. A smaller ℓ corresponds to higher spatial frequencies (rougher profiles), while a larger ℓ results in smoother profiles.

The model was trained on a source distribution defined by $\ell_{\text{train}} = 0.08$. We evaluate the model's generalization capabilities by varying the test correlation length ℓ_{test} across the interval $[0.03, 0.12]$, effectively shifting the spectral density of the input data.

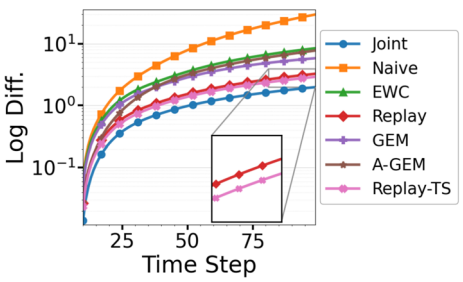


Fig. 18 Error propagation over autoregressive rollout for 1D Burgers. Evolution of the logarithmic difference (Log Diff.) between model predictions and ground truth across the simulation time steps. While baseline methods suffer from rapid exponential error compounding, both *Replay* and *Replay-TS* maintain a lower error profile. *Replay-TS* bounds this error slightly more effectively, which is critical for preventing phase drift over longer horizons

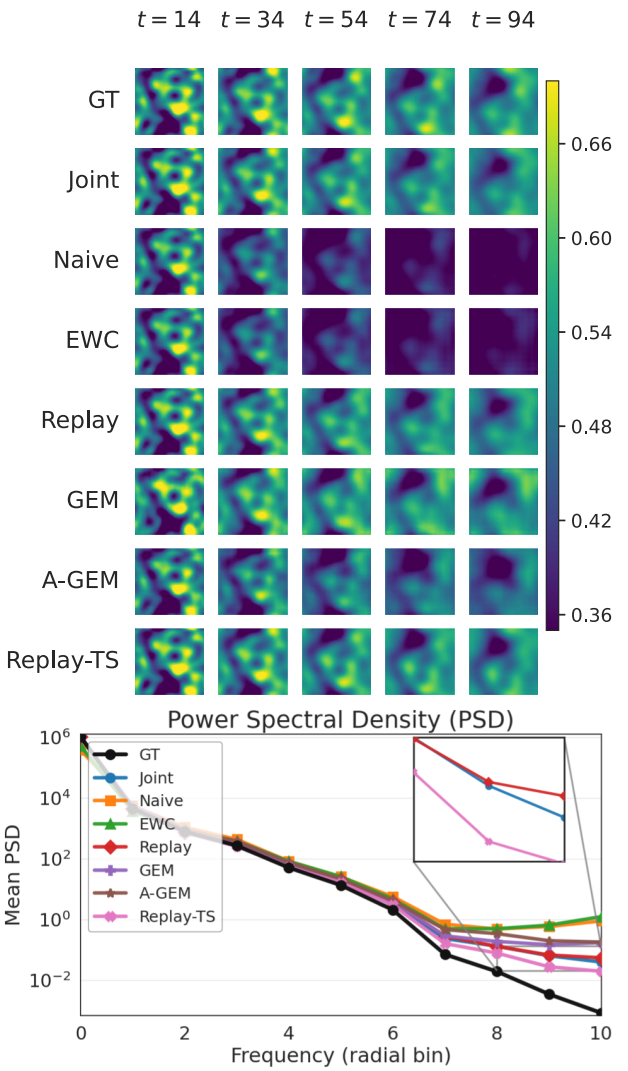


Fig. 19 State evolution and spectral analysis on the 2D advection-diffusion stream. (Top) Qualitative comparison of the scalar field at various time steps. *Replay-TS* shows minimal structural degradation compared to standard Replay. (Bottom) The Power Spectral Density (PSD) plot demonstrates that *Replay-TS* tracks the target frequency distribution closely, avoiding the artificial smoothing (loss of high-frequency energy) evident in the Naive and EWC baselines

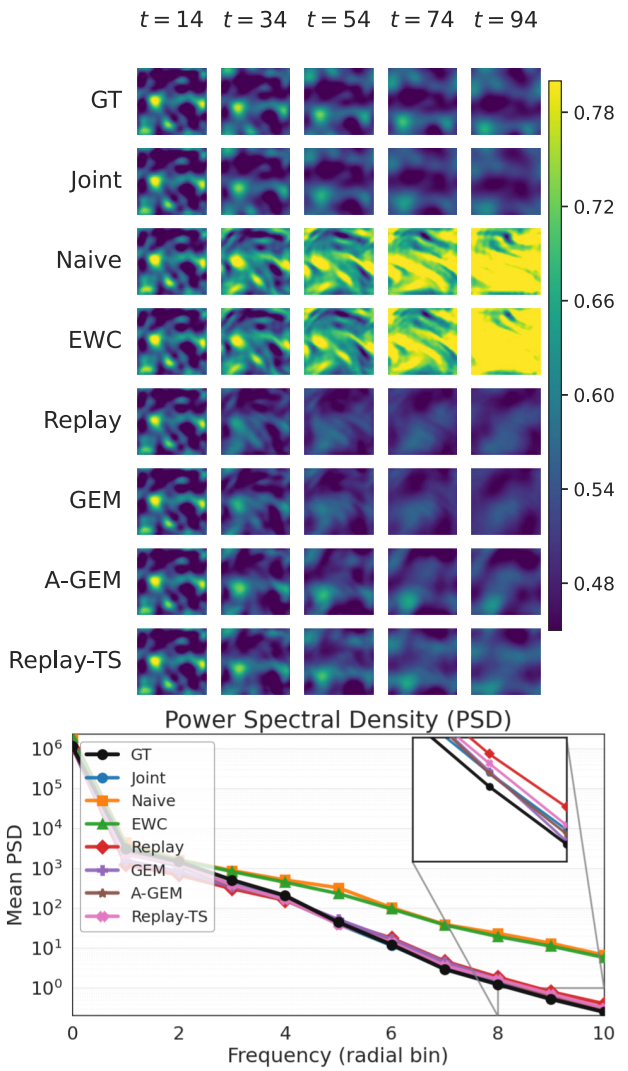


Fig. 20 State evolution and spectral analysis on the 2D mixed physics stream. The model must recall dynamics while alternating between Navier-Stokes and Advection-Diffusion equations. *Replay-TS* successfully preserves a better structural fidelity among the CL methods overall. The PSD plot confirms that *Replay-TS* preserves the broadband energy spectrum more accurately across varying physical regimes

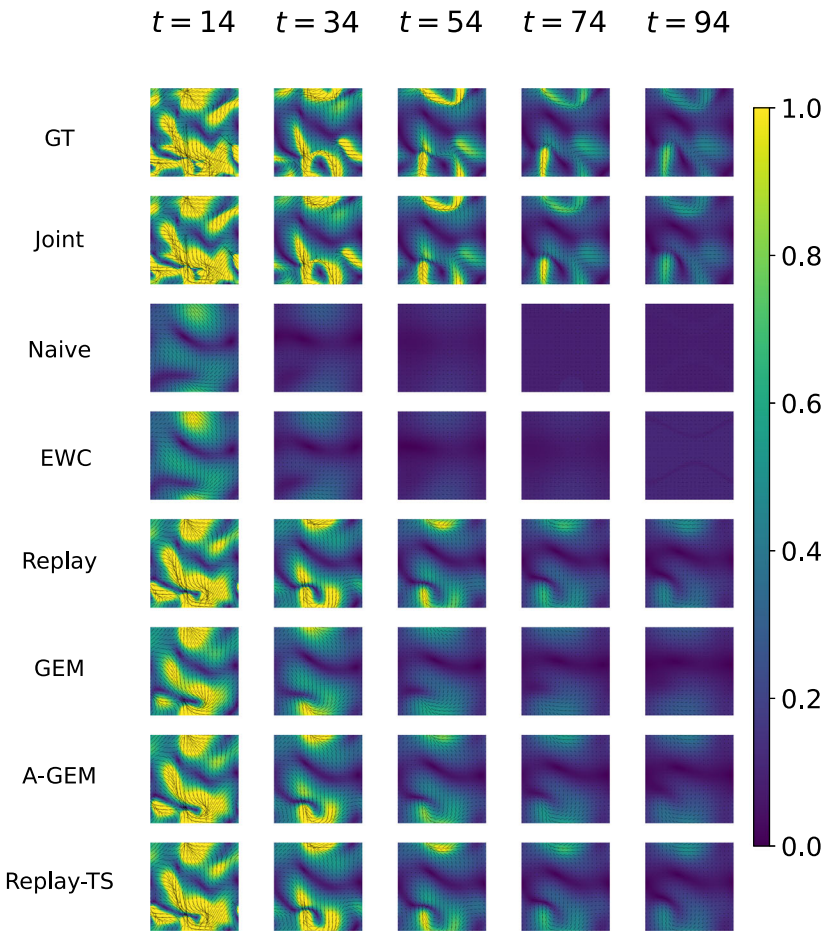


Fig. 21 Qualitative comparison on the 2D Burgers' Stream. Snapshots of the field at various time steps across different CL methods. *Replay-TS* (bottom row) achieves the highest structural fidelity, capturing the sharp, non-linear shock fronts that standard Replay struggles to resolve under strict memory constraints

E.2: Quantitative analysis

Figure 15 illustrates the Relative-L2 Loss as a function of ℓ_{test}. We observe a distinct "U-shaped" generalization curve. The error is minimized at $\ell_{\text{test}} \approx 0.08$, matching the training distribution. Although not as severe as the inter-regime shifts, the performance degrades as the test distribution drifts away from the source regime. Notably, the degradation is asymmetric, which indicates that the model struggles more with lower ℓ values (high-frequency inputs), suggesting that the surrogate fails to extrapolate equally to spatial frequencies not seen during training.

E.3: Qualitative analysis

Figure 16 visualizes the rollout trajectories across different ℓ regimes. The row highlighted in red corresponds to the training regime ($\ell = 0.08$), where the prediction error is negligible. As ℓ decreases (top rows), the initial conditions contain high-frequency oscillations that the model smooths out, leading to significant structural errors. Conversely, at high ℓ (bottom rows), while the dynamics are smoother, the model exhibits phase accumulation errors, confirming that autoregressive operators overfit to the specific spectral density of the training data.

Appendix F: Extended qualitative analyses

In this section, we provide extended qualitative assessments and Power Spectral Density (PSD) analyses to support the quantitative findings presented in the main text. These visualizations serve to further illustrate the specific failure modes of baseline CL methods, as well as the structural improvements yielded by Temporal Slicing (*Replay-TS*).

F.1: 1D Burgers' equation dynamics

Figure 17 visualizes the spatiotemporal evolution of a 1D Burgers' stream sample. The non-linear convective term in

the Burgers' equation naturally leads to the formation of steep gradients and shock waves. While standard Replay recovers the general macroscopic behavior much better than Naive fine-tuning or EWC, it exhibits a slight noisier effect at the shock front. As shown in the corresponding error propagation plot (Fig. 18), the *Replay-TS* method suppresses this error accumulation, maintaining a tighter error bound over extended autoregressive horizons.

F.2: 2D single-physics preservation

The transition to two spatial dimensions introduces more complex frequency distributions. Figure 19 (Advection-Diffusion) and Fig. 21 (Burgers') demonstrate the model's ability to maintain these 2D structures across experiences. The PSD plot in 19 empirically confirms that while Naive and EWC baselines severely damp high-frequency energy (evident by the drop-off in the higher radial bins), *Replay-TS* closely tracks the ground truth spectrum, slightly outperforming standard Replay in preserving these crucial fine-scale physics.

F.3: 2D mixed physics interference

Finally, Fig. 20 illustrates the highly challenging 2D Mixed Physics stream, where the surrogate must alternate between learning linear transport (Advection-Diffusion) and chaotic fluid dynamics (Navier-Stokes). This setting is highly susceptible to "cross-contamination," where the convolutional or spectral filters learned for one PDE inappropriately manifest in the predictions of the other. The visual comparison and accompanying PSD plot demonstrate that *Replay-TS* retains a better structural fidelity for the first experience (which belongs to the 2D Advection-Diffusion equation) after being trained on an experience from the 2D Navier-Stokes equation in the final experience.

Table 14 1D single-physics streams (U-Net). Comparison of CL methods on Advection-Diffusion and Burgers' equations. *Joint* training provides the performance ceiling, while all replay methods are restricted to $S_{\text{total}} = 2000$. *Replay-TS* provides the highest stability, crucial for the U-Net which struggles with translation shifts in the Naive setting. They indicate the lowest value (lower error/loss) in each column, in all cases

Method	Advection-Diffusion		Burgers'	
	MSE	Rel-L2	MSE	Rel-L2
Joint	0.0241 ± 0.0061	0.0332 ± 0.0028	0.0377 ± 0.0045	0.0381 ± 0.0030
Naive	4.3320 ± 0.3659	0.4921 ± 0.0261	2.2916 ± 0.1247	0.3027 ± 0.0042
EWC	3.1423 ± 1.1738	0.3972 ± 0.0515	2.3226 ± 0.2599	0.3167 ± 0.0138
Replay	0.4216 ± 0.0234	0.1462 ± 0.0028	0.0573 ± 0.0031	0.0580 ± 0.0019
GEM	5.2182 ± 0.1159	0.5670 ± 0.0085	0.9137 ± 0.0820	0.1936 ± 0.0101
A-GEM	0.4472 ± 0.0445	0.1624 ± 0.0086	0.1161 ± 0.0092	0.0898 ± 0.0044
Replay-TS	**0.3362** ± **0.0105**	**0.1116** ± **0.0007**	**0.0484** ± **0.0011**	**0.0468** ± **0.0009**

Table 15 2D single-physics streams (U-Net). Benchmarking results for Advection-Diffusion, Burgers' equations, and Navier-Stokes. *Joint* performance represents the upper bound, and replay methods share a memory constraint of $S_{\text{total}} = 2000$. The proposed method consistently outperforms other sequential baselines. They indicate the lowest value (lower error/loss) in each column, in all cases

Method	Adv-Diff		Burgers'		Navier-Stokes	
	MSE	Rel-L2	MSE	Rel-L2	MSE	Rel-L2
Joint	0.0019 ± 0.0002	0.0159 ± 0.0008	0.0157 ± 0.0010	0.0325 ± 0.0017	0.0625 ± 0.0027	0.0083 ± 0.0013
Naive	2.2536 ± 1.2350	0.4528 ± 0.1641	1.1724 ± 0.0942	0.3037 ± 0.0133	1.7806 ± 0.3079	0.2236 ± 0.0373
EWC	0.3570 ± 0.2069	0.1807 ± 0.0643	1.0543 ± 0.1667	0.2624 ± 0.0239	2.2251 ± 0.5900	0.2965 ± 0.0269
Replay	0.0046 ± 0.0004	0.0252 ± 0.0011	0.0391 ± 0.0019	0.0622 ± 0.0012	0.2145 ± 0.0076	0.0170 ± 0.0026
GEM	0.0109 ± 0.0096	0.0342 ± 0.0178	0.1089 ± 0.0099	0.0957 ± 0.0043	0.3186 ± 0.0010	0.0205 ± 0.0010
A-GEM	0.0044 ± 0.0006	0.0214 ± 0.0010	0.0713 ± 0.0056	0.0829 ± 0.0031	0.3324 ± 0.0197	0.0290 ± 0.0018
Replay-TS	$\mathbf{0.0039} \pm \mathbf{0.0007}$	$\mathbf{0.0206} \pm \mathbf{0.0005}$	$\mathbf{0.0322} \pm \mathbf{0.0014}$	$\mathbf{0.0542} \pm \mathbf{0.0012}$	$\mathbf{0.1544} \pm \mathbf{0.0027}$	$\mathbf{0.0155} \pm \mathbf{0.0025}$

Appendix G: U-Net results

To demonstrate that the effectiveness of Temporal Slicing is not limited to spectral architectures like FNO, we present the full benchmarking results using a U-Net backbone. As a Convolutional Neural Network (CNN), the U-Net relies on local receptive fields rather than global basis functions. Consequently, it typically requires a denser sampling of the spatiotemporal manifold to generalize effectively. The results below confirm that the benefits of maximizing buffer diversity via *Replay-TS* are fundamental to the continual learning problem, regardless of the underlying inductive bias.

G.1: 1D U-Net results

Table 14 summarizes the performance on the 1D Advection-Diffusion and Burgers' streams. The U-Net exhibits significantly higher variance in the *Naive* setting compared to the FNO. This is attributed to the convolutional kernels being sensitive to phase shifts caused by changing advection velocities; unlike FNOs, which naturally handle global translations, the U-Net struggles to extrapolate transport speeds without adequate historical context. However, as shown in the table, *Replay-TS* effectively stabilizes the learning process, reducing the MSE in the Advection-Diffusion case to 0.3362, outperforming standard Replay (0.4216).

G.2: 2D U-Net results

The results for the 2D Advection-Diffusion, Burgers', and Navier-Stokes streams are presented in Table 15.

In the Navier-Stokes regime, the memory bottleneck is most acute. *Replay-TS* achieves an MSE of 0.1544 compared to 0.2145 for standard Replay. This confirms that even for architectures capable of capturing fine-scale local features, retaining a diverse set of viscosity regimes via slicing is essential to prevent the model from overfitting to the flow dynamics of the most recent experience.

Appendix H: Computational complexity and runtime analysis

To address the practical deployment considerations of *Replay-TS*, we provide a brief analysis of its computational complexity and empirical wall-clock training times compared to standard baselines.

H.1: Time and memory complexity

When training autoregressive surrogates, the primary computational bottleneck per sequence is the recursive forward pass over the pushforward horizon, L_{pf}. Standard ER must unroll the model for all target steps, resulting in a time complexity of $\mathcal{O}(L_{\text{pf}})$ per rollout.

Replay-TS maintains this exact $\mathcal{O}(L_{\text{pf}})$ time complexity. Because the surrogate must physically propagate the temporal dynamics, it still performs the full autoregressive unrolling. However, because *Replay-TS* sparsely samples the target rollout, it computes the loss and backward gradients only at a subset of steps ($m_\gamma < L_{\text{pf}}$). Consequently, while the forward time complexity remains equal at $\mathcal{O}(L_{\text{pf}})$, the computational graph size during the backward pass is fundamentally reduced compared to standard ER. Furthermore, retaining the specific target indices $\mathcal{K}$ introduces a very small memory footprint, adding only a negligible $\mathcal{O}(1)$ storage overhead per replay item.

H.2: Empirical runtime evaluation

To contextualize the practical computational cost, Table 16 details the total wall-clock training time for the 1D and 2D Advection-Diffusion streams.

As expected, the *Naive* fine-tuning approach is the fastest since it bypasses all memory rehearsal. Standard *Replay* incurs a moderate penalty due to the continuous processing of buffered data, while GEM and A-GEM exhibit higher execution times corresponding to their gradient projection constraints.

Table 16 Empirical wall-clock training time. Total time required to train the FNO backbone sequentially across the full 1D and 2D Advection-Diffusion data streams

Method	1D Adv-Diff (FNO1D)	2D Adv-Diff (FNO2D)
Naive	42m	3h 11m
EWC	1h 02m	4h 38m
Replay	57m	5h 38m
GEM	1h 06m	5h 52m
A-GEM	1h 05m	5h 40m
Replay-TS	1h 12m	6h 34m

In our current benchmarking setup, *Replay-TS* exhibits a slightly longer runtime than standard ER. This overhead is purely an implementation artifact caused by naive, unoptimized index-matching within every mini-batch for the autoregressive rollout loop of our prototype. Transitioning the code to utilize vectorized tensor masking or deterministic modulo arithmetic (for uniform slicing) will eliminate this bottleneck, making *Replay-TS* computationally comparable to, or even slightly faster than, standard ER due to its inherently sparser loss computations.

Author Contributions H.H. developed and implemented the machine-learning methodology, adapted it to the PDE-learning problems and datasets, conducted the analyses and experiments using and extending the provided computational setup where needed, and wrote the initial draft of the manuscript. B.D.N. provided the PDE-learning problem context, prepared the initial simulation datasets and code basis for the main PDE examples, contributed domain expertise, and supported the work through scientific discussions, critical feedback, and manuscript editing. S.S. contributed to the conceptual framing of the study, supervised the project, provided scientific guidance and critical feedback throughout the work, helped shape the manuscript and visual presentation, and acquired funding. All authors reviewed and approved the final manuscript.

Funding Open Access funding enabled and organized by Projekt DEAL. The authors have no funding to acknowledge for this work.

Data Availability The datasets supporting the findings of this work are synthetically generated. The complete data-generation scripts, configuration files, random seeds, training framework, and the proposed Temporal Slicing module are publicly accessible via the Zenodo repository at https://doi.org/10.5281/zenodo.21486100.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Lam R, Sanchez-Gonzalez A, Willson M, Wirnsberger P, Fortunato M, Alet F, et al. Learning skillful medium-range global weather forecasting. Science. 2023;382(6677):1416–21.
2. Merchant A, Batzner S, Schoenholz SS, Aykol M, Cheon G, Cubuk ED. Scaling deep learning for materials discovery. Nature. 2023;624(7990):80–5.
3. Bi K, Xie L, Zhang H, Chen X, Gu X, Tian Q. Accurate medium-range global weather forecasting with 3d neural networks. Nature. 2023;619(7970):533–8.
4. Kochkov D, Smith JA, Alieva A, Wang Q, Brenner MP, Hoyer S. Machine learning-accelerated computational fluid dynamics. Proc Natl Acad Sci. 2021;118(21):2101784118.
5. Sanchez-Gonzalez A, Godwin J, Pfaff T, Ying R, Leskovec J, Battaglia P. Learning to c. In: International Conference on Machine Learning. 2020. p. 8459–68. PMLR.
6. Pathak J, Subramanian S, Harrington P, Raja S, Chattopadhyay A, Mardani M, et al. Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators. 2022. arXiv:2202.11214 arXiv preprint.
7. Allotey J, Butler KT, Thiyagalingam J. Entropy-based active learning of graph neural network surrogate models for materials properties. rrogate models for materials properties. The Journal of Chemical Physics 2021;155(17). https://doi.org/10.1063/5.0065694.
8. Nakka R, Harursampath D, Ponnusami SA. A generalised deep learning-based surrogate model for homogenisation utilising material property encoding and physics-based bounds. Sci Rep. 2023;13.
9. Nguyen BD, Potapenko P, Demirci A, Govind K, Bompas S, Sandfeld S. Efficient surrogate models for materials science simulations: Machine learning-based prediction of microstructure properties. Mach Learn Appl. 2024;16:100544. https://doi.org/10.1016/j.mlwa.2024.100544.
10. Geirhos R, Jacobsen J-H, Michaelis C, Zemel R, Brendel W, Bethge M, et al. Shortcut learning in deep neural networks. Nat Mach Intell. 2020;2(11):665–73.
11. Gulrajani I, Lopez-Paz D. search of lost domain generalization. 2020. arXiv:2007.01434 arXiv preprint.
12. Takamoto M, Praditia T, Leiteritz R, MacKinlay D, Alesiani F, Pflüger D, Niepert M. Pdebench: An extensive benchmark for scientific machine learning. 2022. ArXiv:2210.07182
13. Li Z, Kovachki NB, Azizzadenesheli K, Liu B, Bhattacharya K, Stuart A, Anandkumar A. Fourier neural operator for parametric partial differential equations. In: International Conference on Learning Representations. 2021.
14. Stachenfeld K, Fielding DB, Kochkov D, Cranmer M, Pfaff T, Godwin J, Cui C, Ho S, Battaglia P, Sanchez-Gonzalez A. Learned simulators for turbulence. In: International Conference on Learning Representations. 2022. https://openreview.net/forum?id=msRBojTz-Nh
15. Otness K, Gjoka A, Bruna J, Panozzo D, Peherstorfer B, Schneider T, et al. An extensible benchmark suite for learning to simulate physical systems. 2021.
16. Zong Y, Tartakovsky A. Mathematics of digital twins and transfer learning for pde models. 2025. arXiv:2501.06400 arXiv preprint.

17. Wang Y, Bai J, Eshaghi MS, Anitescu C, Zhuang X, Rabczuk T, et al. Transfer learning in physics-informed neurals networks: Full fine-tuning, lightweight fine-tuning, and low-rank adaptation. Int J Mech Syst Dyn. 2025;5(2):212–35.
18. Balin R, Simini F, Simpson C, Shao A, Rigazzi A, Ellis M, et al. situ framework for coupling simulation and machine learning with application to cfd. 2023. arXiv:2306.12900 arXiv preprint.
19. Meyer LT, Schouler M, Caulk RA, Ribes A, Raffin B. Training deep surrogate models with large scale online learning. In: International Conference on Machine Learning. 2023. p. 24614–30. PMLR.
20. Parisi GI, Kemker R, Part JL, Kanan C, Wermter S. Continual lifelong learning with neural networks: A review. Neural networks the official journal of the International Neural Network Society. 2018;113:
21. Biological underpinnings for lifelong learning machines. Nature. Machine Intelligence. 2022;4:196–210.
22. McClelland JL, McNaughton BL, O'Reilly RC. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. Psychol Rev. 1995;102:3.
23. McCloskey M, Cohen NJ. Catastrophic interference in connectionist networks: The sequential learning problem. Psychology of Learning and Motivation. 1989;24.
24. Lange MD, Aljundi R, Masana M, Parisot S, Jia X, Leonardis A, et al. A continual learning survey: Defying forgetting in classification tasks. IEEE Trans Pattern Anal Mach Intell. 2019;44:3366–85.
25. Mai Z, Li R, Jeong J, Quispe D, Kim HJ, Sanner S. Online continual learning in image classification: An empirical survey. 2021. ArXiv:2101.10423.
26. Kilickaya M, Weijer J, Asano YM. Towards label-efficient incremental learning: A survey. 2023. ArXiv:2302.00353.
27. Rolnick D, Ahuja A, Schwarz J, Lillicrap TP, Wayne G. Experience replay for continual learning. In: Neural Information Processing Systems. 2018. https://api.semanticscholar.org/CorpusID:53860287
28. Willard JD, Jia X, Xu S, Steinbach MS, Kumar V. Integrating scientific knowledge with machine learning for engineering and environmental systems. ACM Comput Surv. 2020;55:1–37.
29. Fish J, Belytschko T. A first course in finite elements. 2007. https://api.semanticscholar.org/CorpusID:118161311
30. Versteeg HK, Malalasekera W. An introduction to computational fluid dynamics - the finite volume method. 2007. https://api.semanticscholar.org/CorpusID:29589033
31. Raccuglia P, Elbert KC, Adler P, Falk C, Wenny MB, Mollo A, et al. Machine-learning-assisted materials discovery using failed experiments. Nature. 2016;533:73–6.
32. Schleder GR, Padilha ACM, Acosta CM, Costa M, Fazzio A. From dft to machine learning: recent approaches to materials science-a review. Journal of Physics Materials. 2019;2
33. Ho J, Jain A, Abbeel P. Denoising diffusion probabilistic models 2020.. ArXiv:2006.11239
34. Goodfellow IJ, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, et al. Generative adversarial networks. 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT). 2021. p. 1–7.
35. Dornheim J, Morand L, Zeitvogel S, Iraki T, Link N, Helm D. Deep reinforcement learning methods for structure-guided processing path optimization. J Intell Manuf. 2020;33:333–52.
36. Barra J, Chahal R, Banerjee S, Pasini ML, Irle S, Lam S. Inverse mapping of properties to composition through generative modeling for designing molten salts. NPJ Computational Materials. 2025;11
37. Wang J, Chen W, Fuge MD, Rai R. Ih-gan: A conditional generative model for implicit surface-based inverse design of cellular structures. 2021. ArXiv:2103.02588.
38. Zeni C, Pinsler R, Zügner D, Fowler A, Horton M, Fu X, et al. A generative model for inorganic materials design. Nature. 2025;639:624–32.
39. Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J Comput Phys. 2019;378:686–707.
40. Cuomo S, Cola VSD, Giampaolo F, Rozza G, Raissi M, Piccialli F. Scientific machine learning through physics-informed neural networks: Where we are and what's next. J Sci Comput. 2022;92
41. Krishnapriyan AS, Gholami A, Zhe S, Kirby RM, Mahoney MW. Characterizing possible failure modes in physics-informed neural networks. In: Neural Information Processing Systems. 2021. https://api.semanticscholar.org/CorpusID:237386144
42. Lu L, Jin P, Pang G, Zhang Z, Karniadakis GE. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. Nat Mach Intell. 2021;3(3):218–29.
43. Kopanicáková A, Karniadakis GE. Deeponet based preconditioning strategies for solving parametric linear systems of equations. 2024. ArXiv:2401.02016.
44. He J, Koric S, Abueidda DW, Najafi A, Jasiuk I. Geom-deeponet: a point-cloud-based deep operator network for field predictions on 3d parameterized geometries. 2024. ArXiv:2403.14788.
45. Pfaff T, Fortunato M, Sanchez-Gonzalez A, Battaglia P. Learning mesh-based simulation with graph networks. In: International Conference on Learning Representations. 2021.
46. Brandstetter J, Worrall DE, Welling M. Message passing neural PDE solvers. In: International Conference on Learning Representations. 2022.
47. Koehler F, Niedermayr S, Westermann R, Thuerey N. APEBench: a benchmark for autoregressive neural emulators of PDEs. 2024. arxiv:2411.00180
48. Holzschuh B, Liu Q, Kohl G, Thuerey N. PDE-transformer: efficient and versatile transformers for physics simulations. 2025. arxiv:2505.24717
49. Menezes AG, Moura G, Alves C, Carvalho A. Continual object detection: a review of definitions, strategies, and challenges. Neural Netw Official J Int Neural Netw Soc. 2022;161:476–93.
50. Wu W, Zhao Y, Li Z, Shan L, Zhou H, Shou MZ. Continual learning for image segmentation with dynamic query. IEEE Trans Circ Syst Video Technol. 2023;34:4874–86.
51. Mermillod M, Bugaiska A, Bonin P. The stability-plasticity dilemma: investigating the continuum from catastrophic forgetting to age-limited learning effects. Front Psychol. 2013;4
52. Kirkpatrick J, et al. Overcoming catastrophic forgetting in neural networks. Proc Natl Acad Sci. 2017;114(13):3521–6.
53. Zenke F, Poole B, Ganguli S. Continual learning through synaptic intelligence. Proc Mach Learn Res. 2017;70:3987–95.
54. Chaudhry A, Dokania PK, Ajanthan T, Torr PHS. Riemannian walk for incremental learning: Understanding forgetting and intransigence. 2018. ArXiv:1801.10112.
55. Aljundi R, Babiloni F, Elhoseiny M, Rohrbach M, Tuytelaars T. Memory aware synapses: Learning what (not) to forget. 2017. ArXiv:1711.09601.
56. Mallya A, Lazebnik S. Packnet: Adding multiple tasks to a single network by iterative pruning. CVPR. 2018. p. 7765–73.
57. Mallya A, Davis D, Lazebnik S. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In: European Conference on Computer Vision. 2018. https://api.semanticscholar.org/CorpusID:3977226
58. Gurbuz MB, Dovrolis C. Nispa: Neuro-inspired stability-plasticity adaptation for continual learning in sparse networks. 2022. ArXiv:2206.09117.
59. Chaudhry A. On tiny episodic memories in continual learning. 2019. arXiv:1902.10486 arXiv preprint.

60. Buzzega P, Boschini M, Porrello A, Abati D, Calderara S. Dark experience for general continual learning: a strong, simple baseline. 2020. ArXiv:2004.07211.
61. Buzzega P, Boschini M, Porrello A, Calderara S. Rethinking experience replay: a bag of tricks for continual learning. In: 25th International Conference on Pattern Recognition (ICPR). 2020. p. 2180–7.
62. Lopez-Paz D, Ranzato M. Gradient episodic memory for continual learning. NeurIPS. 2017.
63. Chaudhry A. Efficient lifelong learning with a-gem. ICLR. 2019.
64. Goswami S. Transfer learning enhanced physics informed neural network for phase-field modeling of fracture. 2019. arXiv:1907.02531 arXiv preprint.
65. Rahman MA, Ross ZE, Azizzadenesheli K. U-no: U-shaped neural operators. Trans Mach Learn Res. 2022.
66. Zhou A, Lorsung C, Hemmasian A, Farimani AB. Strategies for pretraining neural operators. Trans Mach Learn Res. 2024;2024.
67. Shen J, Marwah T, Talwalkar A. UPS: efficiently building foundation models for PDE solving via cross-modal adaptation. 2024. arxiv:2403.07187
68. Jagtap AD, Kharazmi E, Karniadakis GE. Conservative physics-informed neural networks on discrete domains for conservation laws. Comput Methods Appl Mech Eng. 2020;365:113028.
69. Tripura T, Chakraborty S. A foundational neural operator that continuously learns without forgetting. 2023. arxiv:2310.18885
70. Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, et al. Human-level control through deep reinforcement learning. Nature. 2015;518(7540):529–33.
71. Wang L, Xiong Y, Wang Z, Qiao Y, Lin D, Tang X, Van Gool L. Temporal segment networks: Towards good practices for deep action recognition. In: European Conference on Computer Vision. 2016. p. 20–36. Springer

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.